國立政治大學資訊科學系

碩士論文

解耦交易：基於 Kafka 的分散式支付系統

Decoupling Payments: A Distributed Payment System Based on Kafka

指導教授：郭桐惟 博士

研究生: 陳亮酉

中華民國 113 年 3 月

誌謝

首先，我要感謝我指導教授，郭桐惟老師。在入學後的三年間，總是持續不斷的關心著我們的學習進度和研究方向，每周都撥冗與我開會討論。這樣的過程中，讓我收益良多。除此之外，實驗室的同學和學弟妹們也幫了我不少忙。

摘要

本論文利用 Apache Kafka，提出了和以往數位貨幣不同的兩種分散式支付系統的開發模型。我們會著重於通過分散式系統架構和一種新的用戶資金管理模型 (用於提高了平行處理的程度) 來優化性能和可擴展性。在開發的兩種模型中，並行模型因其卓越的能力而脫穎而出，支持每秒超過 200,000 筆交易（TPS），且對硬體要求低，並展現出對網絡挑戰和系統故障的顯著韌性。此研究通過提供一種新方法：通過將交易記錄分段處理，能夠增加交易處理的效率和韌性，同時還能保證銀行系統中的最終一致性。

關鍵字：分散式系統、Apache Kafka、中央銀行數位貨幣（CBDC）、最終一致性

Abstract

This thesis presents the development of two models for a distributed payment system, utilizing Apache Kafka to depart from conventional central bank digital currency systems. The focus is on optimizing performance and scalability through distributed system architecture and a new model for managing user funds which improves the level of parallelism. Among the two models developed, the Concurrent model stands out for its exceptional capability, supporting over 200,000 transactions per second (TPS) with minimal hardware requirements and demonstrating significant resilience to network challenges and system faults. This research introduces a new method: by segmenting transaction records, it can enhance the efficiency and resilience of transaction processing, while also ensuring eventual consistency within the banking system.

Keywords: Distributed System, Apache Kafka, Central Bank Digital Currency (CBDC), Eventual Consistency

Contents

誌謝 i

摘要 ii

Abstract iii

Contents iv

List of Figures vii

List of Tables ix

1 Introduction 1

2 Background 5

2.1 Apache Kafka 5

2.2 Common Fund Representation Models: UTXO & account based 7

2.3 Real-Time Distributed Banking System for Inter-bank Transfers 11

3 System Design 12

3.1 Our Fund Representation: Credit Model 13

3.2 Basic Idea 14

3.2.1 Topics and Partitions 14

3.2.2 Processors 16

3.2.3 Lifecycle of a Trade 17

3.2.4 Logic behind Partition Control 19

3.3 Limitations of the Simplified Design 21

4 Details of the System . . . 23
4.1 Trade Design and Partitioning . . . 23
4.1.1 Partitioning Details . . . 24
4.1.2 Orders Topic and Serializability . . . 26
4.2 Sequential Model . . . 27
4.3 Concurrent Model . . . 29
4.4 Eventual Consistency . . . 31
5 Implementation . . . 33
5.1 Shared Hash map . . . 34
5.2 Choosing Avro Over JSON . . . 37
6 Experimental Results . . . 38
6.1 The Impact of Block Size . . . 39
6.2 The Impact of Kafka parameters . . . 41
6.3 The Impact of Requests Per Second (RPS) . . . 43
6.4 The Impact of Additional Network Latency . . . 48
6.4.1 the Impact of Block Size with Additional Network Latency . . . 48
6.4.2 The Impact of RPS with Additional Network Latency . . . 51
6.4.3 The Impact of Increasing Network Latency . . . 53
6.5 Scalability . . . 55
6.5.1 Scenario 1: Two High-End Machines . . . 56
6.5.2 Scenario 2: Two Lower-Spec Machines . . . 56
6.5.3 Scenario 3: Pairing a High-End with a Lower-Spec Machines . . . 57
6.5.4 Scenario 4: Integrating Two High-End and Two Lower-Spec Machines . . . 59
6.5.5 Analysis of Four Scenarios . . . 60

7 Related Work 62
7.1 Central Bank Digital Currency 62
7.2 Fund Representation 64
7.3 Two-phase Commit 66
8 Conclusion 67
Reference 68

List of Figures

2.1 Account Model Explained 9

2.2 UTXO Model Explained 9

3.1 Credit Model Fund Representation Explained 13

3.2 Lifecycle of a trade 19

3.3 Blocks, Partitioning and Consumer Groups 21

4.1 Sequential model 28

4.2 Concurrent model 30

5.1 Dirty Write Complications in Interlocked and Aborted Transactions . . . 35

5.2 Race Conditions and Their Effects on Non-Atomic Balance Update Operations 36

6.1 Variation of Transactions Per Second (TPS) with Increasing Block Size . 40

6.2 Changes in 95th Percentile Latency with Increasing Block Size 41

6.3 Variation of Transactions Per Second (TPS) with Increasing Max Fetch Bytes 42

6.4 Changes in 95th Percentile Latency with Increasing Max Fetch Bytes . . 43

6.5 Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) 45

6.6 Changes in 95th Percentile Latency with Increasing Requests Per Second (RPS) 45

6.7 Impact of Isolated Transaction Generator for Concurrent Model 47

6.8 Impact of Isolated Transaction Generator for Sequential Model 47
6.9 Variation of Transactions Per Second (TPS) with Increasing Block Size under Simulated Network Delay 50
6.10 Changes in 95th Percentile Latency on a Logarithmic Scale with Increasing Block Size under Simulated Network Delay 51
6.11 Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) under 100ms network delay 52
6.12 Changes in 95th Percentile Latency with Increasing Requests Per Second (RPS)under 100ms network delay 53
6.13 Variation of Transactions Per Second (TPS) with Increasing Network Delay 54
6.14 Changes in 95th Percentile Latency with Increasing Network Delay . . . 55
6.15 Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using Two High-End Machines 56
6.16 Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using Two Lower-Spec Machines 57
6.17 Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using a High-End and a Lower-Spec Machine 58
6.18 Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using Two High-End and Two Lower-Spec Machines . . . 59
6.19 Comparative Analysis of Transactions Per Second (TPS) Variation with Increasing Requests Per Second (RPS) Across Different Machine Configurations 61

List of Tables

1 Introduction

The payment system is essential for banks to handle numerous transactions, including deposits, withdrawals, transfers, and remittances. These transactions are carried out through various channels, such as ATMs, online banking, or over-the-counter services at local banks. However, numerous transactions involve multiple banks. For instance, consider transfers between different banks or withdrawals from ATMs that do not belong to the account holder's bank. Therefore, a system that facilitates continuous transaction processing between banks and provides the results (whether the transaction was successful or not and the bank balances after the transaction) to the banks is necessary.

The payment system between banks must efficiently process a high volume of transactions every second, ensuring the proper handling of each transaction to prevent double spending. While many countries have relied on centralized systems for decades, the availability of open-source distributed systems has fueled a growing interest in developing distributed payment systems between banks to achieve enhanced throughput.

Research into enhancing banking systems with distributed technologies faces significant challenges [1, 2, 3, 4]. Prior studies have predominantly relied on UTXO-based or account-based models for fund representation, leading to widespread debates [5, 1].

This thesis introduces a hybrid approach that enables parallel processing without strictly adhering to traditional account-based or UTXO-based systems, combining the benefits of account-based systems with improved throughput under the structure of streaming.

The primary challenge lies in preventing dirty writes and ensuring data consistency during parallel processing. A distributed payment system can provide better scalability and potentially enhance throughput. However, achieving parallel processing in such a system introduces the challenge of ensuring that related transactions are processed correctly. Specifically, we encounter challenges when dealing with transactions involving the same account, whether as the receiver or sender. If not handled properly, parallel processors may unintentionally use the same previous balance as the last balance, leading to conflicts and, in some cases, only one transaction being processed successfully.

To illustrate, consider an account with a balance of $100 involved in two transactions: one gaining $5 and the other $10. After these transactions, if both are successfully completed, the balance should end up as $115. However, two processors might both start with the account's initial balance of $100 and subsequently output new balances of $105 and $110. In this scenario, the last recorded state will be either $105 or $110, neither of which is the correct final balance.

To address conflicts arising from transactions involving the same accounts, we have explored several solutions. The most straightforward approach would be to assign transactions with the same accounts to the same processor.However, it's important to note that we are dealing with both the sender and receiver of a transaction. Ensuring that certain accounts never interact with others is impossible and unrealistic for a banking system, making this approach nonviable to be distributed. Another potential solution involves using locks to manage conflicts, which is the traditional account-based method. However, implementing locks can significantly increase the number of steps required to complete a transaction.

To bypass the issue of account locks, which could significantly raise latency, we've developed a two-phase strategy. First, we categorize transactions based on their sender banks (the banks from which funds are transferred) and ensure that a bank's balance is

maintained by only one processor at a time. Since every transaction involves two participants, one representing the sender bank (which pays) and the other representing the receiver bank (which gains), we categorize transactions according to the sender bank involved. Secondly, we decouple transactions into two stages: adding and deducting money. After validating a transaction, we send the validated data on how much money the account should add if the transaction is validated to the relevant processor for processing. This approach allows us to efficiently manage and process transactions while maintaining data consistency.

In this thesis, we built a distributed payment system based on Apache Kafka [6]. Given that our system operates through multiple phases and essentially acts as a streaming system, Apache Kafka, the leading open-source stream-processing software, emerges as a suitable option for our implementation. Furthermore, its impressive scalability, fault tolerance, and support for KStream [7] and KSQLDB [8], making it an ideal choice for constructing real-time systems. Apache Kafka also features a built-in two-phase commit (2PC) method, which is essential for achieving atomic writes. Given our primary objective of creating a secure and efficient real-time streaming system, Apache Kafka stands out as a valuable tool to help us meet these goals.

In Chapter 2, we will delineate the foundational concepts pertinent to Apache Kafka, distributed payment systems, and the predominant data representation methodologies in distributed payments, namely the Unspent Transaction Output (UTXO) and account-based models. Chapters 3 and 4 are dedicated to articulating our principal hypothesis, followed by an exposition of the two models we developed to address distinct scenarios. Chapter 5 delineates various implementation challenges encountered during the development of these models. Chapter 6 presents the outcomes of our experiments. Initially, we assess several key parameters to ascertain a comprehensive understanding of the system's tuning logic. Subsequently, we conduct throughput and latency evaluations under varying con-

ditions, including alterations in the number of machines and the introduction of network delays. Chapters 7 and 8 compile related works, facilitating comparative analyses, and culminate in a synthesis of the entire thesis.

2 Background

In this chapter, we furnish the essential background pertinent to this thesis. This includes a comprehensive introduction to our fundamental platform, Apache Kafka, and an exploration of works related to distributed payment systems. Secondly, we engage in a detailed discussion contrasting the UTXO-based transaction models with their Account-based counterparts. Finally, we introduce the real-world scenarios of our concern and explain how to design a system that fits these scenarios.

2.1 Apache Kafka

Apache Kafka is an open-source distributed streaming platform based on event-driven and publish-subscribe mechanisms. It is designed to handle large-scale real-time data streaming in a distributed and fault-tolerant manner. Kafka is known for its high throughput, scalability, and integration with other data processing systems such as Hadoop, Spark, and Flink, making it a backbone for real-time data processing pipelines. Apache Kafka comprises four primary components: topics, brokers, producers, and consumers. Each plays a vital role in the ecosystem:

1. Topic:

   Topics in Kafka act as categories for messages and can be divided into partitions to facilitate parallel message processing. Each topic encompasses one or more parti-

tions. By default, data is distributed across these partitions in a round-robin fashion. However, to imbue partitions with specific characteristics or to designate the purpose of each partition, manual data allocation can be implemented.

2. Partition and Offset:

   Within these partitions, data is stored as offsets, ensuring complete serialization within a single partition. This organization guarantees that offsets within the same partition are chronologically ordered based on their entry into Kafka. Conversely, offsets in different partitions of the same topic do not necessarily maintain a clear temporal relationship. In the context of our research, both the default and customized partition methodologies are employed. The default method helps mitigate potential unpredicted traffic congestions, while specifying partition usage is instrumental in constructing a robust parallel architecture.

3. Cluster and Broker:

   Brokers are the servers that make up the Kafka cluster. They store and replicate messages, manage topic partitions, and handle requests from producers and consumers. Multiple brokers are needed to increase throughput and ensure basic fault tolerance through replica management. Furthermore, like other distributed systems, Kafka has an advantage over centralized systems. It can use multiple brokers to build geographically distributed systems.

4. Producer:

   Producers write messages to topics, and consumers read messages from topics. Producers send messages to specific topics and can optionally specify a partition key.

5. Consumer:

Consumers subscribe to one or more topics and consume messages from them. Consumers can also specify which partitions they want to read from and manage their own offsets to keep track of the messages they have read. In order to read and process multiple partitions of a topic, consumers need to have a group to ensure that the same record (or offset in Kafka) is never read twice.

6. Zookeeper:

   Apart from Kafka itself, Zookeeper [9] is an auxiliary system that is often used alongside Kafka. It is a distributed coordination service and often used to manage and coordinate the Kafka brokers and make up the Kafka cluster. Zookeeper maintains configuration information, provides leader election (of partitions), and manages consumer groups.

2.2 Common Fund Representation Models: UTXO & account based

Before delving into the intricacies of our system's design, it is imperative to decide on the method of representing users' funds within the system. The landscape of distributed payment systems and cryptocurrencies predominantly features two transaction models. This section provides a concise overview of both, highlighting their unique mechanisms and applications.

1. UTXO Model:

   The first model, known as the Unspent Transaction Output (UTXO) model, operates on the principle of spending outputs from prior transactions to generate new outputs, which, in turn, can be utilized in future transactions. This approach emphasizes the

tracking of discrete funds, allowing for a granular control over transactions. In practice, users manage multiple UTXOs, combining them as necessary to facilitate transactions, receiving any excess as new UTXOs. This model is employed by Bitcoin as well as a multitude of other blockchain platforms [10, 11].

2. Account Model:

   Conversely, the account model simulates a traditional bank account, focusing on the tracking of balances rather than discrete funds. Transactions are approved based on the sufficiency of the account's balance to cover the proposed expenses. This model, which underpins Ethereum [12], streamlines the transaction process and is purported to offer superior support for smart contracts, as discussed in the Ethereum whitepaper. The account model's simplicity and efficiency make it a compelling choice for systems requiring complex interactions and contractual agreements.

The concepts of the account and UTXO models are illustrated in Figure 2.1 and 2.2 using the simple example where Alice sends $10 to Bob. The account model atomically validates and processes the transactions between the two accounts. Conversely, the UTXO model first selects and deducts a UTXO from Alice with sufficient funds, and then creates a new UTXO for Bob.

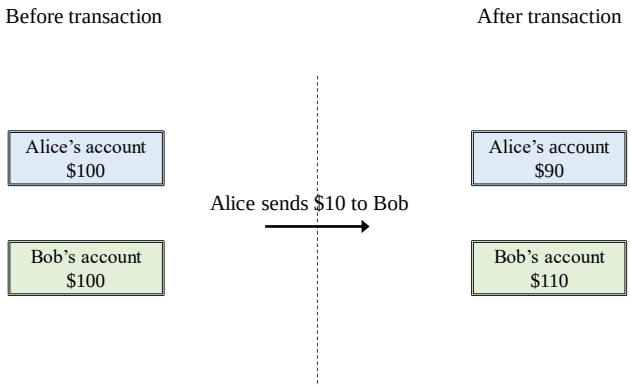


Figure 2.1: Account Model Explained

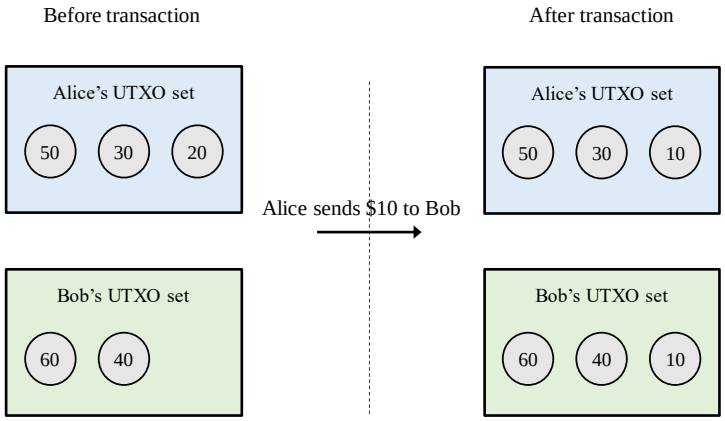


Figure 2.2: UTXO Model Explained

Both models present distinct advantages and considerations. The UTXO model's emphasis on transaction outputs offers a high degree of security and traceability, making it ideal for systems prioritizing transparency and security. On the other hand, the account model's straightforward balance tracking facilitates easier management and integration

of smart contracts, appealing to platforms aiming for versatility and scalability. This dichotomy underscores the necessity of selecting a model that aligns with the system's objectives, ensuring optimal performance and user experience.

In considering our specific requirements, our objectives do not center on transparency, security, or smart contract capabilities. Instead, our primary focus is on achieving high performance alongside robust consistency assurance. Despite the discussion above, the UTXO model exhibits superior compatibility with parallel processing, offering notable advantages in this regard. Conversely, the traditional account model streamlines transactions by eliminating the need to verify every fragment of funds within the same account during validation. Given these considerations, we have tailored a model that more closely aligns with our unique needs. The details of this customized approach will be elaborated upon in Chapter 3.

We will not conduct an actual experiment to test the efficiency of our models against the two common fund representation models. When comparing to the UTXO model, there are many restrictions in real applications. To compare our models with it on the same basis, we would need to follow specific rules or machine setups, which would be unfair to our models. As for the Account model, its framework is significantly different from our streaming structure, making it very difficult to design fair experiments that evaluate all models on the same basis. Thus, our discussion of the UTXO model and Account model will conclude here in theory.

2.3 Real-Time Distributed Banking System for Inter-bank Transfers

An obvious question will occur: if you cannot prove the benefits of your models comparing the traditional ones, why design them? The answer is: our models are designing for some specific situations.

We are considering inter-bank transfers within a country, with users sending requests individually (not facilities sending massive requests via scripts). We found that a streaming system is suitable for creating a real-time distributed system that is safe, fast, and low-cost. The UTXO model might be too complex for this simple scenario, and the Account model is not suitable for real-time cases.

In the following two chapters, we will introduce how we design real-time distributed banking systems for inter-bank transfers using Kafka, and discuss the benefits of these systems and their streaming structure.

3 System Design

In this chapter, we introduce the credit model, the fund representation framework at the heart of our thesis. This model aims to amalgamate the strengths of both the account balance and UTXO models to create a system that is eventually consistent and optimized for distributed banking operations. To maximize the benefits of distributed computing, we have developed the credit model with a specialized approach to topic partitioning within Apache Kafka.

We then detail the core architecture of our system, encompassing the design of Kafka topics and partitions, the roles of processors, and a comprehensive workflow. Additionally, we explain the rationale behind our partition control design.

In conclusion, we discuss the challenges and limitations of our preliminary design, setting the stage for the detailed exploration of two advanced models in Chapter 4, which represents the primary focus of this thesis.

It is worth noting that what are commonly referred to as 'transactions' in other distributed systems or blockchain contexts are also termed 'trades' within our system. This distinction is made to avoid confusion with Kafka's own transactional mechanisms, which are crucial for achieving exactly-once semantics in our system's foundational structure. By using 'trades' to describe account-to-account payments, we clearly differentiate between our payment data and Kafka's transaction terminology."

3.1 Our Fund Representation: Credit Model

As previously delineated, there exist two predominant models for representing funds within a system: the account balance model and the Unspent Transaction Outputs (UTXO) model. The account balance model, while providing a straightforward portrayal of balances, necessitates the locking of accounts during trades, potentially decelerating the process. Conversely, the UTXO model facilitates enhanced trade processing parallelism but does not conveniently aggregate account balances.

In our research, we propose an new methodology for account management in distributed banking systems, termed the ”credit model.” Analogous to the UTXO model's generation of UTXOs, our model introduces a new data element upon the partial completion of a trade. Figure 3.1 shows the concept of the credit model.

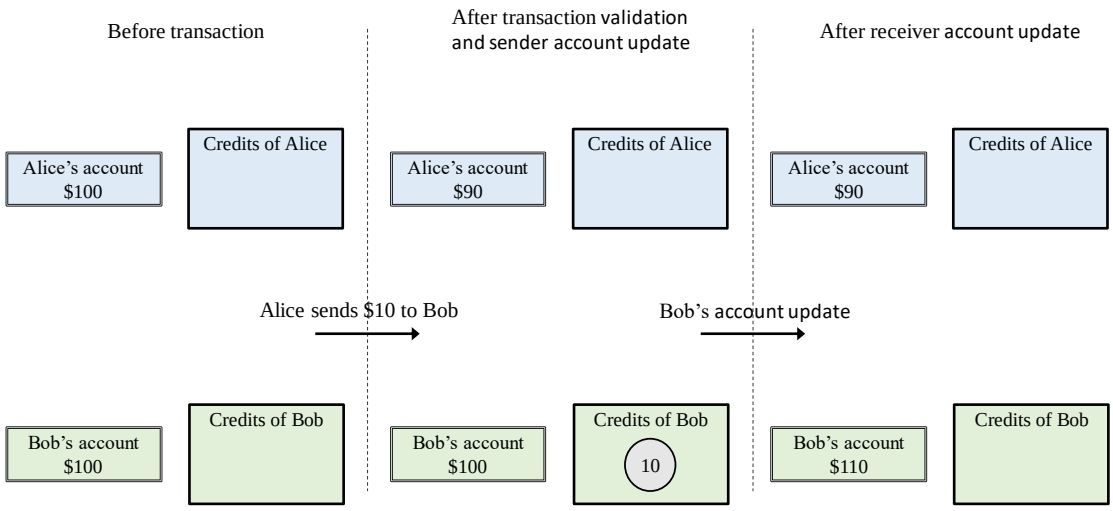


Figure 3.1: Credit Model Fund Representation Explained

This mechanism not only reduces the potential overhead associated with the account model during concurrent trades involving the same account but also facilitates rapid credit-

to-balance reconciliation, ensuring balances' eventual consistency.

Furthermore, a challenge in the UTXO model, where trades exceeding the account balance necessitate either direct trade rejection or the use of multiple UTXOs—thereby increasing processing time—does not occur in our credit model thanks to the incorporation of "accounts." This feature ensures smoother trades without the need for multiple UTXOs, streamlining the process.

In summary, our credit model combines key features of both account balance and UTXO models. It maintains a continuous balance for each account, similar to the account balance model, while adopting the UTXO model's strategy of requiring validation for only a portion of each trade—the outgoing segment. Unlike the UTXO model, which creates a new UTXO upon trade validation, our approach generates a "credit" item. This credit is eventually incorporated into the recipient's account balance, streamlining the trade process without compromising on security or efficiency.

3.2 Basic Idea

In this section, we will outline the fundamental concepts and workflows of our systems. We have developed two distinct systems, the specifics and differences of which will be detailed in Chapter 4. However, the core principles underlying both systems are consistent. In the following sections, we will outline the architecture of our systems and guide you through the trades' lifecycle. This discussion will include the logic behind partition control within Kafka topics, which will also be described in detail.

3.2.1 Topics and Partitions

Our system utilizes four primary Kafka topics: *Trades*, *Blocks*, *Credits*, and *Orders*, in addition to several minor topics dedicated to state storage. For most of these topics,

the number of partitions is set to match the number of processors. This configuration represents the maximum level of parallelism achievable within our system.

1. Trades: Each trade is initially sent to the *Trades* topic, where the number of partitions is configured to align with the count of first-layer processors—referred to as *aggregators*. It is important to note that the data contained within this topic is irregular

2. Blocks: Data processed by the aggregators is categorized according to the sender account's bank. This data is subsequently consolidated into blocks to reduce the frequency of transmissions before being dispatched to the *Blocks* topic. Moreover, the configuration of partitions within the *Blocks* topic is adjusted to match the number of second-layer processors, referred to as *validators*.

3. Credits: As previously mentioned, we employ a distinct approach, known as the credit model, to manage incoming funds. The credits, representing money that has yet to be added to the account, are stored in the *Credits* topic. The configuration of partitions in this topic matches that of the *Blocks* topic, reflecting their shared polling by the validators. It is noteworthy that in one of the subsequent system, this topic is eliminated. This decision is based on the realization that the functionalities it served can be handled by the Trades and *Blocks* topics.

4. Order: After processing the original trades data, irrespective of whether the validation outcome is successful or rejected, we assign tags to them and dispatch these records to the *Order* topic. Additionally, we send records after the credits have been processed to signify the conclusion of the trades. The number of partitions in this topic is also configured to match the number of *validators* to enhance the parallelism of data writing, although this is not strictly necessary.

3.2.2 Processors

In our system, processors play a dual role, acting both as producers and consumers, similar to other streaming systems. They are tasked with retrieving data, processing it, and then forwarding the processed data onwards. Our system employs a two-layered processor architecture to manage these tasks efficiently.

1. aggregators:

   In our experiments and practical applications, trades are initially not sorted by their origin, which brings into play another critical component: the *aggregators*. *Aggregators* continuously retrieve data from Kafka and fulfill two main functions.

   The first function is to act as classifiers. As we aim to organize trades from the same sender bank within the same partition of the *Blocks* topic, *aggregators* are crucial in achieving this organization.

   Besides classification, *aggregators* also bundle trades into blocks prior to dispatch. This method draws inspiration from blockchain technology [13], which places a premium on preserving a consistent global order of trades by grouping them into blocks before they undergo validation. This strategy is particularly beneficial as it addresses the inefficiency of fetching and sending data from Kafka for each separate trade, an issue that becomes pronounced under conditions of high network latency. By consolidating trades, we not only diminish the frequency of data transmissions to Kafka but also reduce the number of times *validators* need to retrieve data from Kafka. While there are additional configurations related to consumer polling that will be discussed in Chapter 6, aggregation is a necessary step to minimize data transmission frequencies.

2. validators: For the second layer of our processing architecture, the primary objective

is to validate each trade, determining whether it should be successful (indicating the sending account has sufficient remaining balance) or rejected. This functionality underpins the designation of these processors as validators.

Each time a trade reaches a validator, we look at the sender's account balance, make a decision, and send back the result to Kafka. For this process, it's important that we keep a record of the each account's balance right inside the *validators*.

However, a challenge arises in recording the comprehensive change of a trade during validation if the balance of the receiving account is managed by a different validator. To address this, we utilize the credit model: if the receiving account's balance is maintained by another validator, the validating processor issues a credit to Kafka. Concurrently, validators are tasked with continuously monitoring for updates in credits, paralleling the validation process.

3.2.3 Lifecycle of a Trade

let's use the same example of which Alice send $10 to Bob to breakdown our system. Fig 3.2 showns the lifecycle of the trade which Alice send $10 to Bob walk through the system.

At Fig 3.2.a, the client, might be an owner of an account or a bank, send generate and send the trade into our system. after that, one of the aggregator(s) in the system will consume that trade and wrapped it with other trades which shared the same sender bank with Alice and send the block to the concerned partition in Block topic(Fig 3.2.b).

To fully complete the trade, both validators of Alice's and Bob's banks will be included (suppose that Alice's and Bob's account doesn't shared a validator). Fig 3.2.c is the validation step, the validator of Alice's bank will check If Alice has a sufficient balance to cover a $10 payment and send the result to *Orders* topic, while if the balance is

sufficient, also send a credit to Credits topic at the same time. The receiver account update (Fig 3.2.d) is more simple than the previous step since the the validation is finished and we just need to add the credit to account without checking anything. At last, a record is sent to the *Orders* topic to signify that the trade is fully completed.

Let's simplify and clarify the explanation using the same example where Alice sends $10 to Bob, to illustrate how our system operates. The following Fig 3.2 breaks down the lifecycle of a trade as it progresses through our system:

In Fig 3.2.a, a client, who could be either an account holder or a bank, in this case, Alice, initiates and sends the trade into our system. Subsequently, one of the system's aggregators picks up this trade. This aggregator groups it with other trades originating from the same sender's bank as Alice's and forwards this bundled block of trades to the appropriate partition within the *Blocks* topic (as shown in Fig 3.2.b).

To fully complete the trade, validators for both Alice's and Bob's banks are involved (assuming Alice and Bob do not share a validator). As depicted in Fig 3.2.c and Fig 3.2.d, the first phase represents the validation step. The validator associated with Alice's bank verifies whether Alice has enough balance to make a $10 payment. The outcome of this validation is then reported to the *Orders* topic. If Alice has sufficient funds, a credit for the trade is also sent to the Credits topic concurrently. The second phase involving updating the recipient's (Bob's) account balance is straightforward. Since the validation process has concluded, the credit is simply added to Bob's account without further checks. Finally, a record is dispatched to the *Orders* topic to mark the trade as fully completed.

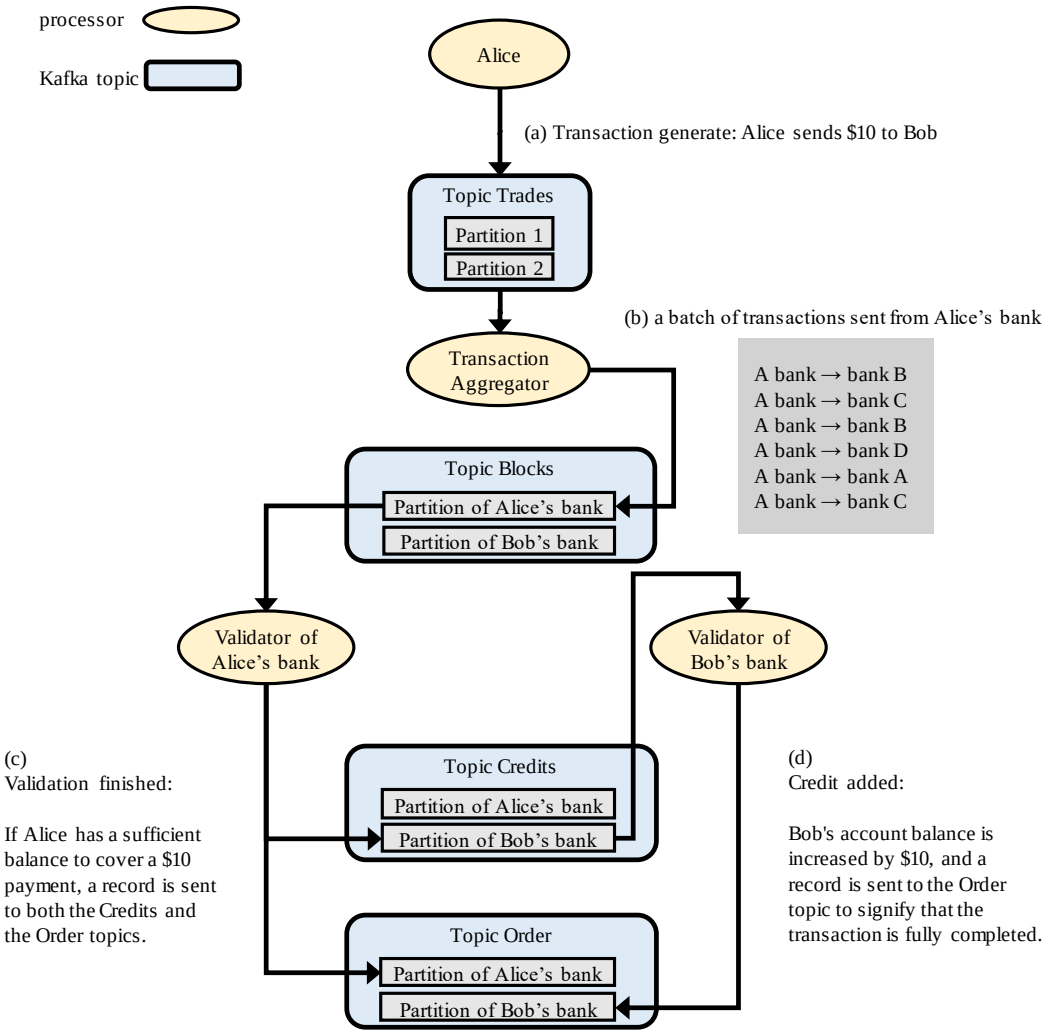


Figure 3.2: Lifecycle of a trade

3.2.4 Logic behind Partition Control

Beyond the fundamental components, we implement a mechanism for maintaining state consistency by persistently writing the most recent account balances to a dedicated Kafka topic named *localBalance*. This topic serves as a repository for the latest balance states of all accounts. In the event of a processor failure or when a new processor is introduced to enhance parallel processing capabilities, each processor within the Kafka

consumer group reassigns partitions and accesses the *localBalance* topic to refresh its state information.

When validators process trades, depending solely on local account balances for calculations can lead to inaccuracies, especially if another validator has already processed a trade for the same account but the balance update is pending. To avoid this without any protective measures, validators must refresh the account balance by querying the *localBalance* topic in Kafka before each trade, whether it involves crediting or debiting an account. However, this method can substantially increase latency due to the need for frequent balance refreshes.

We comes out with an answer by manipulating the Kafka partition. addressing the fundamental issue by ensuring that a single validator consistently manages each account balance, whether for deductions or credits, could be a solution. We allocate each bank to a partition in the *Blocks* topic and group all trades involving the same sender account in the same partition. The concepts of partitioning and consumer groups are depicted in Figure 3.2.

We devised a solution by leveraging Kafka partition manipulation to address the core issue, ensuring a single validator manages all deductions and credits for the same account. By assigning each bank to a specific partition within the *Blocks* topic and grouping all trades from the same sender account into the same partition, we enhance consistency and efficiency. It also leverages Kafka's consumer group feature, sharing metadata among consumers in the same group to allocate partitions and prevent redundant reads. The mechanisms of blocks, partitioning, and consumer groups are illustrated in Figure 3.3.

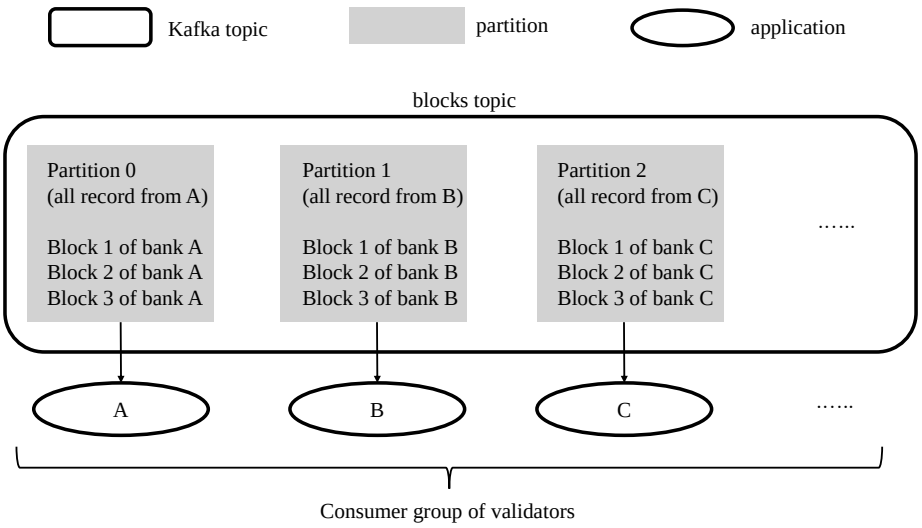


Figure 3.3: Blocks, Partitioning and Consumer Groups

For incoming funds, the models that will be discussed in Chapter 4 take two distinct approaches. One model uses a direct method by reallocating credit data to the *Blocks* topic, transitioning it from the sender's to the receiver's partition, effectively addressing the issue.

Conversely, the second model leverages the *Credits* topic, as illustrated in Figure 3.2. After validating a trade, validators record it in the *Credits* topic, which is organized according to the receiver's bank. This setup enables validators to track the *Credits* topic for updating account balances, mirroring how they handle outgoing *Trades* with the *Blocks* topic.

3.3 Limitations of the Simplified Design

In Chapter 4, our systems will be presented following the conceptual framework outlined in Figure 3.2, with a focus on the system's architectural perspective. Both systems introduce a new topic dedicated to storing the states of account balances, a detail not dis-

cussed in this chapter. The primary distinction between the two models lies in their handling of credit records. One model adheres closely to the structure depicted in Figure 3.2, while in the other, validators retrieve credit records directly from the *Blocks* topic after these credits have been rerouted to the *Trades* topic post-validation.

The issue of consistency in distributed systems, not addressed in the chapter's description, will be explored subsequent to the presentation of the two models. This discussion will clarify the level of consistency achievable within these models.

4 Details of the System

In this chapter, we begin by explaining the design of our payment system. To ground the subsequent discussion in concrete terms, we will delve into the design of our data. This includes an exploration of the critical attributes and some of the logic behind their design.

Subsequently, we will detail the two models we have developed: the Concurrent model and the Sequential model. These models were conceived based on the principles outlined in Chapter 3. We will illustrate their workflows, demonstrating how a payment is processed within each system. This detailed comparison will provide insights into their operational nuances and the rationale behind their design, ensuring a comprehensive understanding of their functionalities and the scenarios they are best suited for.

At the conclusion of the chapter, we will show that although we cannot achieve the standard consistency typical of conventional account-based models, we are able to ensure eventual consistency in both models presented in this thesis. In scenarios without traffic congestion, the result we attain comes very close to that of the conventional models.

4.1 Trade Design and Partitioning

Trades within our systems include various crucial attributes, including the sending and receiving accounts, the Kafka partitions of the sending and receiving banks, the category of the trade, and the amount, among others.

Some attributes play a role in the validation or credit addition processes, such as the sending account, receiving account, and the trade amount. Others determine the appropriate Kafka topic partition for processors to target post-processing, influenced by factors like the account, bank, or category. The processing-related attributes are straightforward, but the latter category, affecting partitioning decisions, involves more complexity, meriting further discussion.

Following that, we will engage in a brief discussion on consistency, as our system also maintains the serial number and various timestamps in each trade record. These elements highlight the inherent characteristics of our system.

4.1.1 Partitioning Details

Let's begin by examining the attributes related to the sending accounts and the sending banks. Building upon the design principles set forth in Chapter 3, to avoid dirty reads and writes during parallel processing, it is important to categorize trades by account. This ensures that all incoming or outgoing trades for a given account are processed by the same validator.

The methodology utilizing Kafka for managing trades faces an issue with directing trades that share common characteristics (such as those transferring money to the same account) to the same Kafka partition within specific topics (notably, the *Blocks* and *Credits* topics). While the smallest unit for grouping trades is based on their associated account, it is impractical to set the number of partitions to match the total number of accounts. Creating a separate partition for each account would require generating thousands of partitions, leading to an increased load on the brokers. This, in turn, would compromise the efficiency of consumer operations, such as polling from Kafka brokers, and adversely affect the system's scalability.

Optimally, the number of partitions should match the number of consumers in a consumer group. Given that the payment system's architecture is envisioned to support geographical distribution across banks nationwide, the ultimate aim is to develop a distributed payment system among banks. Ideally, each bank would host its own set of brokers, aggregators, and validators in various locations. Simplifying the scenario by setting aside large banks with a significantly higher volume of accounts and trades compared to smaller regional banks, we assume a uniform scale among banks, with each possessing a broker, an aggregator, and a validator at its headquarters.

The most effective strategy for partitioning trades involves grouping trades related to the same bank within the same partition, enabling each bank to process its respective partition with its validator. While bank-based grouping is not mandatory, it represents a strategic approach to aligning partition numbers with consumer counts in the validation consumer group, allowing for flexibility in grouping strategies under different circumstances.

After validation

The category attribute also deserves careful consideration. Initially categorized as " send, " a payment ' s category is updated to " receive " upon successful validation, and the transaction is then requeued in Kafka as credit. Although this categorization is not essential in the Concurrent model, it is vital for the Sequential model.

As we discuss the attributes of the sending accounts and sending banks, it's important to note that their role in partitioning is taken over by the receiving accounts and receiving banks once the trades have been validated.

4.1.2 Orders Topic and Serializability

Furthermore, we maintain the serial number, the payment amount, and various timestamps for each trade, which are forwarded to the final topic in our system—the *Orders* topic. The design of our payment system focuses on parallel processing of original and credit portions of trades, yet it encounters limitations, particularly evident in the resulting *Orders* topic. To ascertain the validation timing of payment orders, records should be sent post-validation, even though the trades may not be fully complete. Conversely, to compile fully completed payment orders, messages should be dispatched after credit processing, which would significantly delay real-time reporting—a critical aspect for banking operations. Ultimately, we opted to transmit records to the *Orders* topic after both the original and credit trades have been processed separately. Our system thus embodies an eventually consistent distributed framework, utilizing credit records to indicate the timing and sequence of fully completed payments.

When discussing system ordering, assessing the feasibility of achieving linearizability or serializability becomes essential. Due to the division of trades into groups for parallel processing, achieving linearizability is unattainable. However, serializability is achievable, provided we overlook the potential for negative account balances.

Although inter-bank serializability may not be as critical as intra-bank serializability, another option involves segmenting the *Orders* topic into bank-specific partitions, similar to the *Blocks* topic. This approach sacrifices inter-bank serializability in favor of enabling parallel final writes into the *Orders* topic by validators.

4.2 Sequential Model

The first model we introduce is termed the sequential model, depicted in Figure 4.1. This design closely aligns with traditional UTXO-based banking systems, characterized by its straightforward approach. It involves the transmission of credit records back to the ”trades” topic within Kafka, where they are subsequently aggregated into blocks alongside other credit and original trade records.

The process unfolds as follows:

A trade initiates and is dispatched to the ”trades” topic within Kafka (as illustrated in Figure 4.1.a). Aggregators, serving as the initial layer of processors, consistently monitor the ”trades” topic. Upon detection, a trade record is selected by an aggregator (Figure 4.1.b). These aggregators consolidate records associated with the same bank or account group into blocks, which are then transmitted to the *Blocks* topic (Figure 4.1.c). Subsequently, Validators, constituting the second layer of processors, extract data from the *Blocks* topic. Here, trades are categorized and prepared for validation (Figure 4.1.d).

If the initiating account of the trade does not have enough funds, the system generates a duplicate of the trade, sends it to the *Orders* topic, and marks the trade as rejected (Figure 4.1.f). Conversely, if the trade is validated (indicating the originating account possesses adequate funds for the trade), the validator segregates the trade, directing the credit portion back to the ”trades” topic and the validated trade record to the *Orders* topic (Figure 4.1.e and f), while concurrently updating the sender’s account balance in local variables and backing up this data on the Kafka server (Figure 4.1.g). In the ensuing stage, a validator responsible for the recipient’s account processes the credit segment of the trade (Figure 4.1.d), which necessitates no further verification and can be directly added to the recipient’s account balance. Following the addition of funds to its local variables, the validator

dispatches the trade's concluding segment to the *Orders* topic(Figure 4.1.f), signaling the completion of the trade, and updates the Kafka backup accordingly (Figure 4.1.g).

The workload imposed on validators significantly surpasses that of aggregators, particularly due to the employment of Kafka trades to facilitate atomic writes, which also represents the primary bottleneck within the system. Our objective is to streamline the validators' tasks, emphasizing efficiency in their polling activities from the *Blocks* topic.

This model's approach to managing credit data involves directing credit information to the "trades" topic during the validation phase. This credit data is subsequently treated akin to original trade data, amalgamated into blocks, and forwarded to the *Blocks* topic. This eliminates the necessity for resource segregation or alternating the consumption topic post an interval.

Consequently, data within the *Blocks* topic encompasses both original trade and credit data. This consolidated method simplifies data retrieval, requiring only access to the *Blocks* topic.

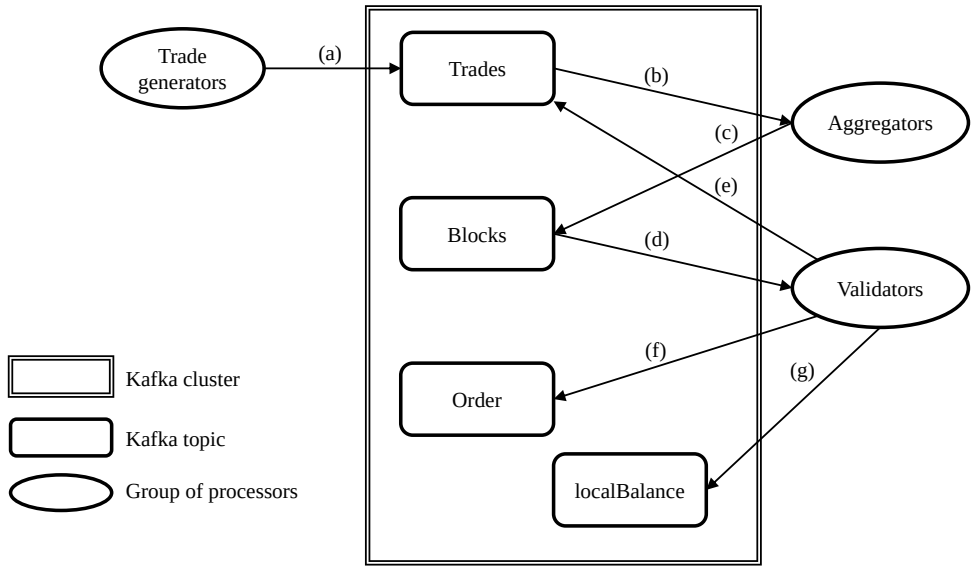


Figure 4.1: Sequential model

4.3 Concurrent Model

In our pursuit to enhance the system's capability to achieve eventual consistency of trades more swiftly, albeit at the expense of some throughput, we devised an alternative approach termed the ”concurrent model.” This model is distinguished by its parallel processing of original and credit records, as depicted in Figure 4.2.

The procedural steps in this model largely mirror those of the previous model, with a notable divergence occurring at Figure 4.2.e. Unlike the previous approach, where credit data was dispatched to the ”trades” topic and later retrieved from the *Blocks* topic (as illustrated in Figures 4.1.e and d), in the concurrent model, credit records are sent directly to *Credits* topic. These records are specifically directed to the partition associated with the recipient's bank (or the respective group of accounts), facilitating consumption by validators as outlined in Figure 4.2.e.

A critical distinction of this model is the validators' operational change; rather than drawing data from a single topic as in the previous model, the concurrent model necessitates simultaneous data retrieval from two distinct Kafka topics. Initially, an attempt was made to accomplish this through a singular thread that would alternate connections between the *Blocks* and *Credits* topics at set intervals (e.g., every 10 seconds). However, frequently establishing connections with the Kafka server proved to be inefficient due to the overhead of establishing connections to the Kafka server.

To address these challenges, we adopted a dual-thread strategy, allowing for concurrent data consumption from the two Kafka topics. This modification introduced additional complexity to the system. For example, it became imperative to synchronize the partitions of both topics to ensure they corresponded to the same set of accounts or, for the case of our experiment, the same banks. Furthermore, this approach raised concurrency issues

due to shared variables between threads, which will be elaborated upon in the subsequent chapter.

Furthermore, by enumerating the steps from the initial source producer, through the generation of credit record, to the final dispatch of the credit record to the *Orders* topic—each requiring data transmissions between clients and brokers—network latency emerges as a potential bottleneck, particularly if the brokers involved are not within the same Local Area Network (LAN). Upon analysis, it is evident that the Sequential model necessitates a total of 9 steps to complete the process, whereas the Concurrent model requires only 7 steps. This distinction underscores the impact of architectural choices on the efficiency and responsiveness of the system, with the Concurrent model offering a streamlined process that potentially reduces the latency introduced by network delays.

In conclusion, by segregating the processing resources into two distinct streams for the *Blocks* and *Credits* topics, our design not only accommodates credit records in a separate queue from original trades but also enhances system flexibility under conditions of instability or significant network delays.

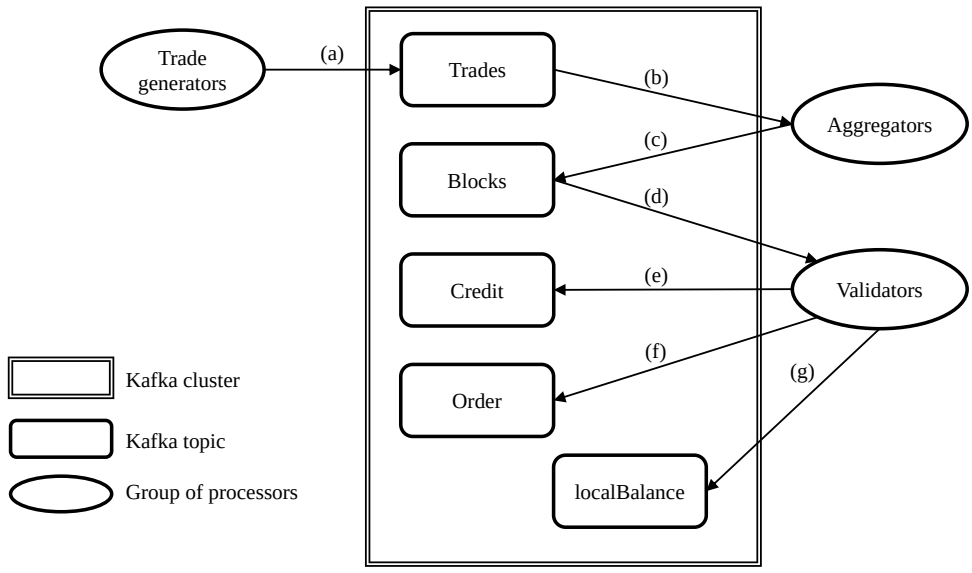


Figure 4.2: Concurrent model

The theoretical advantages and limitations of both models are acknowledged, and their efficiency across various scenarios will be evaluated. Chapter 6 is dedicated to comparing and contrasting these models, providing a comprehensive analysis of their performance differences in specific contexts.

4.4 Eventual Consistency

Addressing consistency issues is paramount in the design of distributed systems, even in the absence of malicious attacks. For instance, node failures during operational hours pose significant challenges. It is crucial to ensure that the system can either reset without generating incorrect outcomes or continue functioning seamlessly thereafter. Faults may arise even without node failures, such as records being overlooked or executed multiple times. To prevent any trade from being ignored, it is vital to ensure that every record is sent and acknowledged at least once at every step within the system. Conversely, to avert double-spending, it is necessary to guarantee that each trade is processed no more than once.

Attempting to resolve all consistency issues independently is impractical; hence, our reliance on Kafka to mitigate the majority of these challenges. This reliance on Kafka is a principal reason for developing our system on this platform. Kafka's architecture, with its replication and consumer group mechanisms, allows for continued processing even after some brokers and processors fail. Through Kafka transactions, we achieve exactly-once semantics by combining idempotent and atomic writes, ensuring both at-least-once and exactly-once processing.

To maintain Kafka's consistency safeguards, we avoid direct interactions between processors. Inter-thread communication within the same processor also presents risks, as identified during the implementation of one of our models (detailed in Chapter 5). There-

fore, we channel all inter-layer processing through Kafka. Adhering to Kafka's protocols effectively addresses consistency concerns, albeit with a slight impact on efficiency, which is a trade-off for ensuring consistency.

While employing the credit model, achieving immediate consistency remains out of reach; we cannot attain the standard consistency typical of conventional account-based models. However, we can ensure eventual consistency in both models presented in this thesis. This does not mean we are at a disadvantage. In scenarios without traffic congestion, the results we achieve are very close to those of conventional models. By accepting a slight delay in achieving consistency, we significantly increase the level of parallelism.

5 Implementation

We developed the applications for both the concurrent and sequential models in Java. Though numerous language-specific clients are available for interfacing with Kafka brokers. Our preference for Java was driven by two main factors: firstly, Apache Kafka itself is developed in Java and Scala, operating on the Java Virtual Machine (JVM). Secondly, Kafka's native API, also written in Java, simplifies programming against Kafka clients significantly. This means developers can avoid writing extensive low-level code to develop functional applications that interact with Kafka.

Regarding our Kafka setup, we utilized the Confluent Platform, a comprehensive data streaming platform that encompasses Kafka. The decision to employ the Confluent Platform was primarily due to its user-friendly schema registry. Given our payment data type requires encapsulating various attributes, while JSON is a viable option, we discovered that Avro offers a more suitable solution for our needs. The Confluent Platform's support for schema registry integrated with Kafka made it an obvious choice for our implementation.

The remainder of this chapter will discuss several intriguing issues we encountered during the system's implementation.

5.1 Shared Hash map

In the Concurrent model, we employ two threads within the validator to independently poll from the *Blocks* and *Credits* topics. Initially, we anticipated this setup would parallelize the process and significantly enhance throughput. However, discrepancies were observed in the *Orders* topic, indicating issues with dirty reads and race conditions, despite the transactional guarantees provided by Kafka.

Upon further investigation, we considered the use of Java's concurrent hash map, believing it might suit our needs. Unfortunately, this turned out to be a misconception. We encountered two main issues:

Firstly, updating account balances within Kafka transactions can lead to dirty reads. We structured the system so that two threads poll their respective topics' specific partitions independently—one thread processes a transaction indicating account A is to pay $10, while the other processes a credit of $10 to account A. These Kafka transactions typically proceed smoothly. However, issues arise if, for instance, network delays cause an acknowledgment timeout, leading to one Kafka transaction being aborted, potentially resulting in dirty writes as illustrated in figure 5.1.

Ideally, the balance should adjust to 90 if the transactional write by Thread-0 is aborted. To ensure the *localBalance* topic on Kafka reflects the most current balances, we concurrently update this topic and the balances hash map. Nonetheless, issues emerge when the last committed *localBalance* record includes a transaction from TopicPartition1 that ought not to be committed and is slated for reprocessing after a ROLLBACK.

While theoretically possible to rollback the *localBalance* record later, this interim period leaves our backup inaccurate. More critically, if the transaction abortion triggers or results from a thread crash, reverting the erroneous write becomes unfeasible, compound-

ing the challenge of maintaining consistent state information in distributed systems.

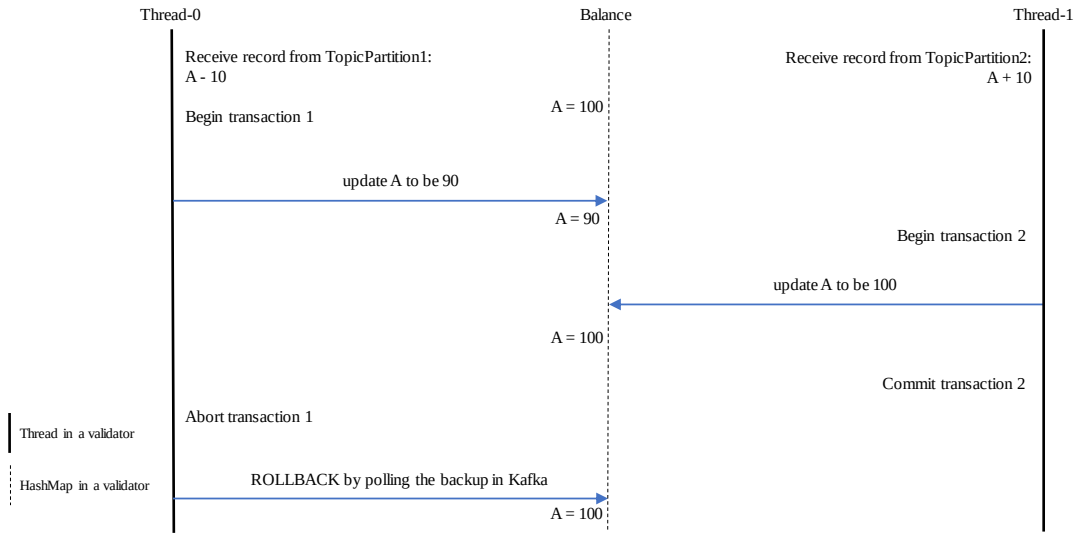


Figure 5.1: Dirty Write Complications in Interlocked and Aborted Transactions

In the second scenario, opting to update account balances after completing Kafka transactions introduces race conditions when updating the *localBalance* Kafka topic, which serves as a backup for account balances. Despite the transaction, both threads need to verify and replicate the balance from the hash map during the Kafka transactional write to calculate the new balance, presenting a significant risk to data consistency.

Imagine the same scenario as before, with two threads processing their respective payments independently. To circumvent the issue of dirty writes identified earlier, we decide to update the hash map post Kafka transaction, as depicted in figure 5.2.

Ideally, the balance should remain at 100, but if one thread replicates the balance hash map before another thread has the chance to update it, the recorded balance in the topic could erroneously show as 90 or 110, neither of which is accurate. This situation underscores the persistent challenge of race conditions, exacerbated by the absence of locks and the non-atomicity of updates between the hash map and the *localBalance* topic.

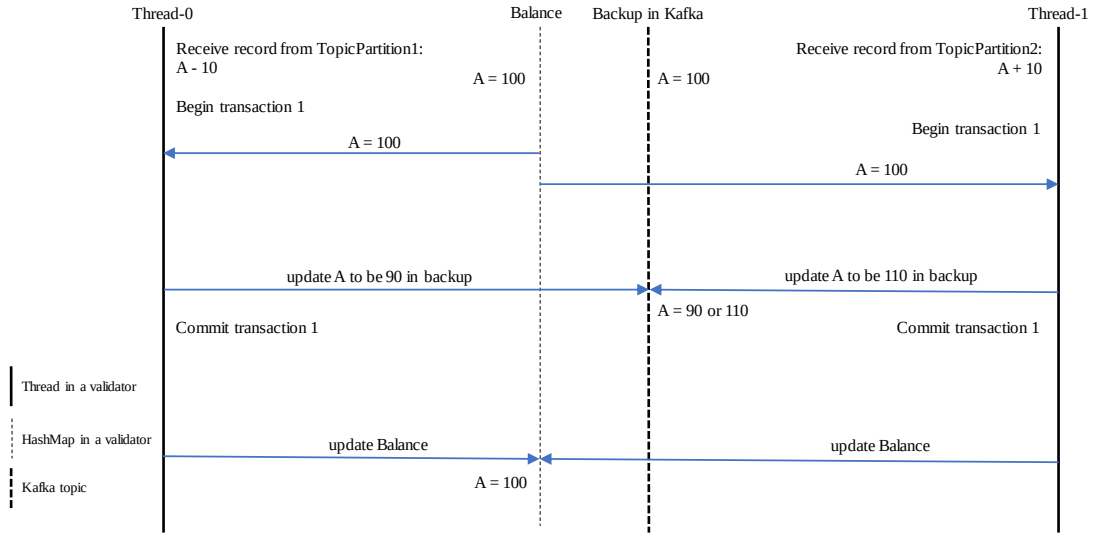


Figure 5.2: Race Conditions and Their Effects on Non-Atomic Balance Update Operations

Therefore, to enable the sharing of hash maps in our setup, we had to concede that parallel processing within the validator is impractical. Our ultimate approach reverts to the traditional method of locking the hash maps during use (within a Kafka transaction).

We eventually termed this approach the Concurrent model, as it involves two threads sharing resources and hash maps. This configuration does not enhance processing speed or throughput. However, there are positive aspects to consider. Although the model does not support parallel processing within the validator, it optimizes the handling of credit records in several ways.

Firstly, credits are no longer relegated to the end of the payment queue, which typically follows unvalidated original payments. This adjustment reduces the latency between validation and the completion of credit processing, particularly beneficial when the Requests Per Second (RPS) significantly exceed the Transactions Per Second (TPS) capacity of the system.

Secondly, the number of steps required to process the *Credits* topic is reduced. In

Chapter 6, which explores tests involving network delays, we will demonstrate that the Concurrent model exhibits improved performance as network delay increase.

5.2 Choosing Avro Over JSON

In developing our system, selecting the appropriate data serialization format was crucial. After a comprehensive evaluation, we opted for Avro instead of JSON, due to its significant advantages in enhancing our system's performance and scalability.

Avro's binary serialization format offers substantial benefits in terms of efficiency. Contrary to JSON, which incorporates both data and its structure in a text-based format, Avro serializes data without embedding the schema. This approach markedly reduces payload size, a critical factor in high-throughput environments characterized by extensive data transmission. Additionally, Avro's architecture promotes quicker serialization and deserialization, thereby boosting system responsiveness and overall performance.

Another notable aspect is schema evolution—a feature where Avro particularly excels over JSON. Avro provides robust support for schema evolution, facilitating effortless updates to data structures without impacting ongoing operations. This adaptability is vital for the sustained maintenance and evolution of our system, ensuring compatibility across different versions. Despite not planning to utilize schema evolution at the current stage, the mere presence of this feature and the necessity of a schema registry could introduce unnecessary complexity for our specific use case.

To sum up, our preference for Avro is primarily due to its efficiency and compact data serialization capabilities. Although Avro's support for schema evolution and the associated requirement for a schema registry might seem superfluous for our immediate needs, the overall performance and compactness benefits make Avro the superior choice for our system.

6 Experimental Results

Our experiments were mostly performed on two identical machines running Ubuntu, each equipped with an Intel® Core™ i7-12700H CPU. On each machine, we hosted a Kafka broker along with a suite of applications that included both an aggregator and a validator.

In the first three sections, we focused on tuning the system's parameters to deepen our understanding and identify optimal general settings. We examined the Transactions Per Second (TPS) and latency at the 95th percentile of transactions changes while increasing block size, one of the Kafka parameters—Max Fetch Bytes, and Requests Per Second (RPS).

Subsequently, maintaining the same machine configuration, we embarked on a comprehensive network delay simulation. We experimented with varying the block size and RPS under certain fixed network delays to determine the system's maximum throughput capacity. Additionally, we tested under incrementally increasing the network delay to observe its impact on system performance trends. Although our simulation approach—introducing delay through the network interface card—may not perfectly replicate real-world distributed system challenges, it remains a valuable tool for identifying potential issues and understanding the distinctive characteristics of our system. Through this process, we aim to uncover insights into how network latency impacts the system's efficiency and identify areas for improvement.

Finally, we expanded our experimental setup to include four machines: In addition to the primary setup, our experiments were supported by two lower-specification machines, each powered by an Intel® Core™ i7-10510U CPU. This processor is two generations older than the i7-12700H and features only 4 cores, which is 10 fewer than the i7-12700H. This diverse array enables us to experiment with different machine combinations, providing valuable insights into the system's scalability. By experimenting with various configurations of these machines, we aim to glean insights into the scalability of our system. This phase is critical for understanding how our distributed payment system adapts to different hardware capabilities, thereby providing a holistic view of its scalability and operational flexibility across diverse machine configurations.

6.1 The Impact of Block Size

The determination of block size within our distributed payment system is influenced by multiple factors beyond the mere parameter of block size itself. The frequency of incoming payments per second and the total number of aggregators in operation are significant determinants of how effectively blocks are formed.

To address periods of low transaction activity and prevent the issue of underutilized block capacity, a timeout mechanism has been instituted. This mechanism, activating after ten seconds, ensures that payments are aggregated into blocks even during intervals of reduced transaction volume, thereby maintaining operational efficiency across varying levels of system activity.

A notable aspect of our system architecture is the dynamic allocation of transactions to aggregators. Transactions originating from the same group of accounts are not funneled to a specific aggregator based on pre-defined criteria. Instead, they are assigned to aggregators on a random basis. Consequently, a single aggregator may be responsible for

creating multiple blocks, each associated with different groups of accounts, within any given ten-second window.

In summary, within a single aggregator, the block size parameter signifies the upper limit on the number of transactions from the same group of accounts that can be aggregated into a block over a designated ten-second period. This approach underscores our system's adaptability and responsiveness to fluctuating transaction volumes, ensuring consistent performance and efficiency.

In our subsequent experiment, we evaluated the performance of the two models by configuring the Requests Per Second (RPS) to 100,000 and progressively increasing the block size from 100 to 3,000. To ensure that the influence of previously mentioned parameters, such as the maximum poll bytes and maximum poll numbers, was negligible, these were adjusted to sufficiently high values. The outcomes of this experiment are illustrated in Figure 6.1 and 6.2.

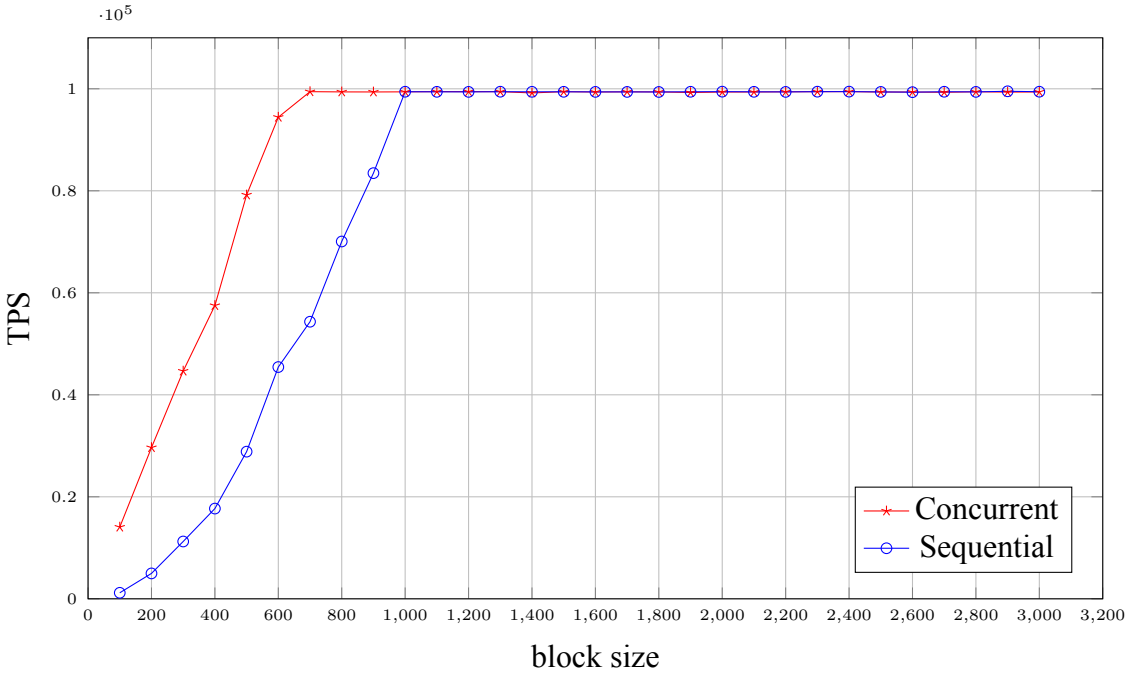


Figure 6.1: Variation of Transactions Per Second (TPS) with Increasing Block Size

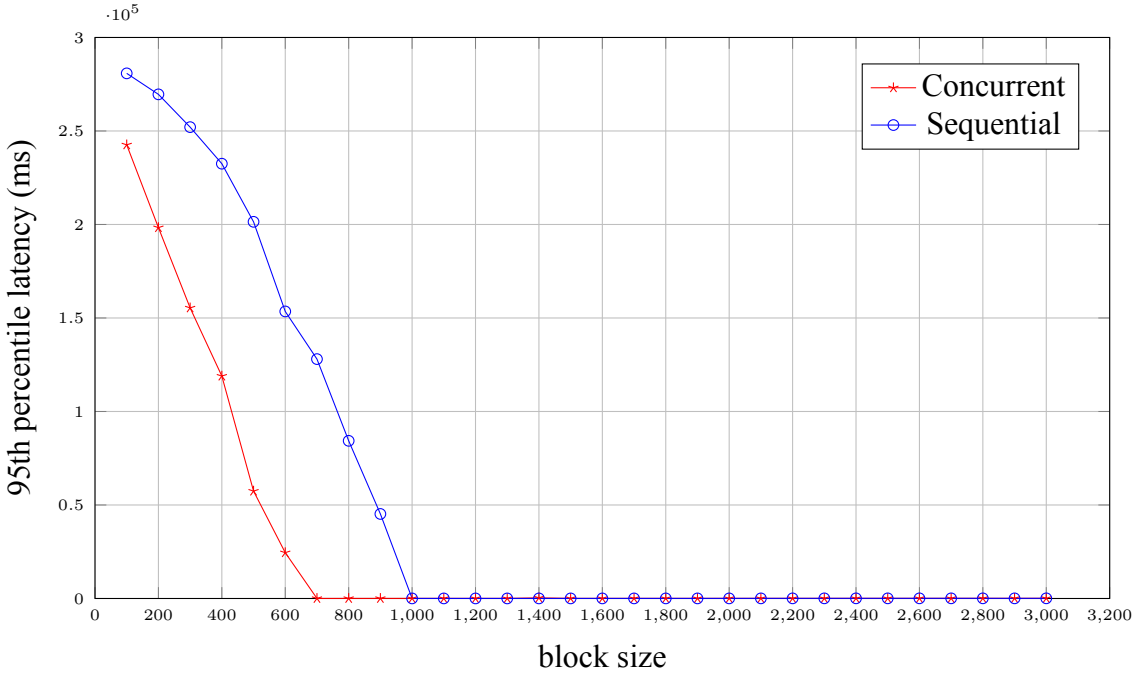


Figure 6.2: Changes in 95th Percentile Latency with Increasing Block Size

6.2 The Impact of Kafka parameters

Many parameters can be adjusted while using Kafka. Of course, many of them can impact the efficiency of a system. Among all, the parameters controlling the number of records affect us the most.

In refining our experimental approach, we identified that adjusting the maximum number of records per poll and the maximum byte size of each poll yield similar effects on our system's performance. However, for our specific use case, controlling the maximum byte size of each poll proved to be more practical. After the initial processing layer, payments are aggregated into blocks, resulting in block sizes that can be significantly larger—by hundreds or thousands—than the size of individual payments. To maintain clarity and avoid the complexity of simultaneously changing multiple parameters in our experiments, we standardized all consumers within the system to have a very high maximum number of records per poll. We set the block size at 3,000 and focused solely on incrementally

increasing the maximum byte size of each poll in scenarios with a Requests Per Second (RPS) rate of 300,000, as depicted in Figure 6.3 and 6.4.

This methodological simplification allowed us to isolate the impact of the maximum byte size of each poll on system performance. Our findings revealed that setting the maximum byte size too low can substantially constrain the system's capacity.

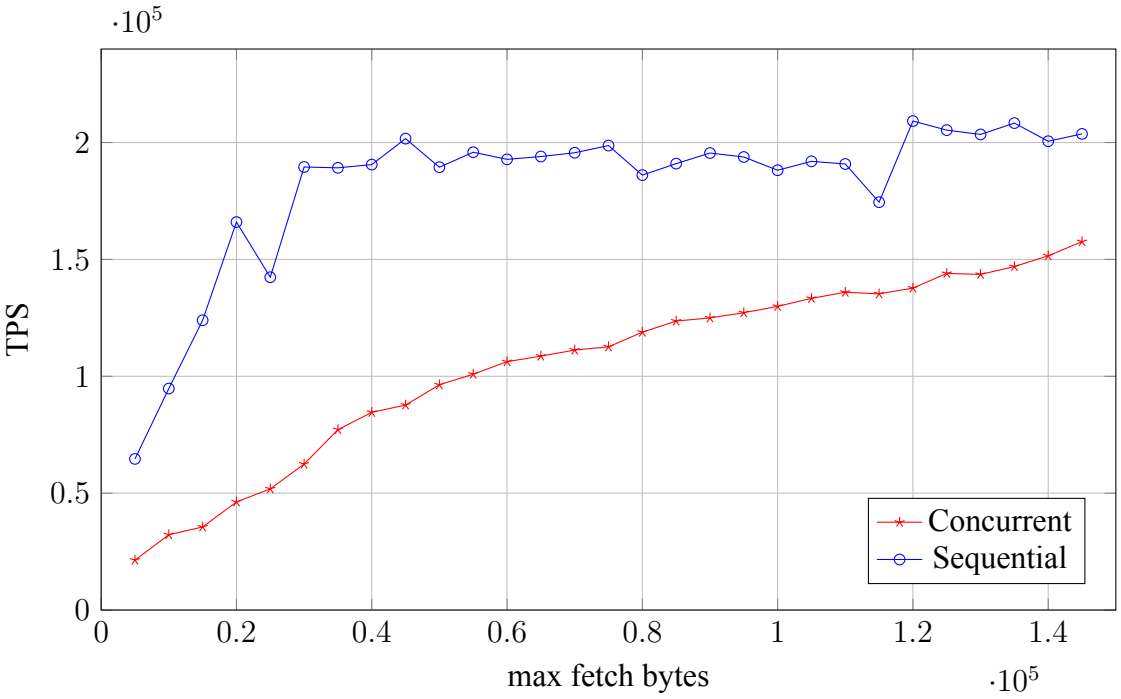


Figure 6.3: Variation of Transactions Per Second (TPS) with Increasing Max Fetch Bytes

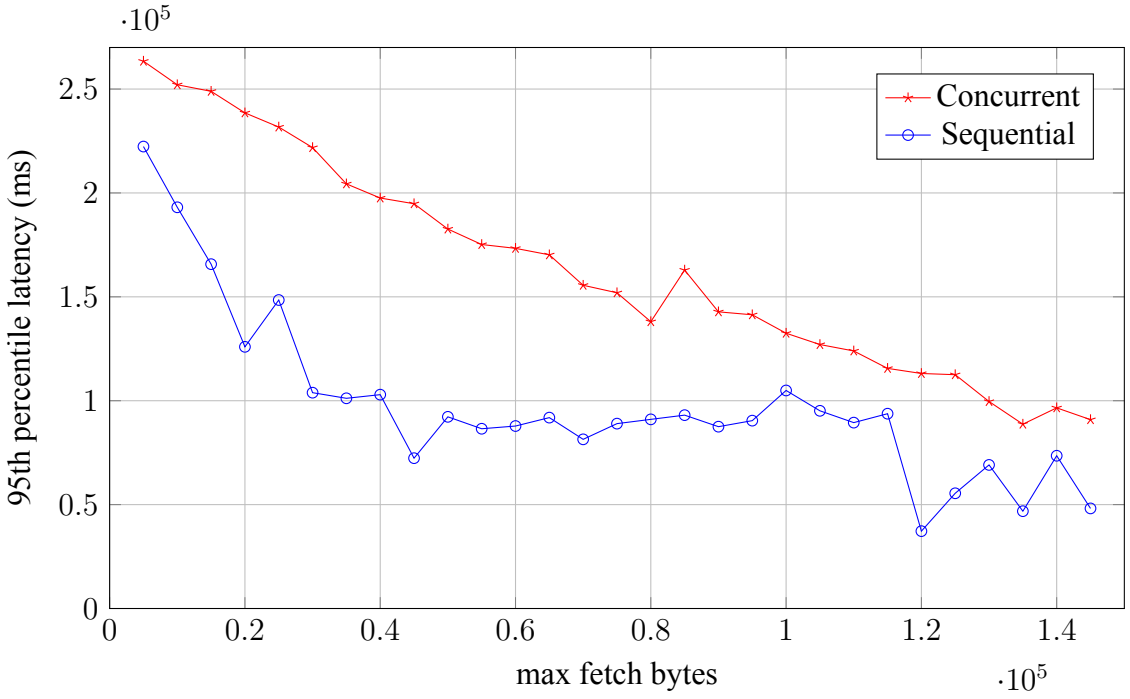


Figure 6.4: Changes in 95th Percentile Latency with Increasing Max Fetch Bytes

6.3 The Impact of Requests Per Second (RPS)

In our study, we evaluated the performance dynamics, specifically Transactions Per Second (TPS) and 95th Percentile Latency, by progressively increasing the Requests Per Second (RPS) to ascertain the limits of our systems. In addition to the Concurrent and Sequential models, we introduced a variant of the Concurrent model that operates without Kafka transactions to serve as a baseline. It is crucial to note that operating the Concurrent model without Kafka transactions poses significant risks to consistency; however, this experimental setup was chosen to quantify the impact of employing Kafka transactions on system performance.

The findings, as depicted in Figure 6.5 and 6.6, reveal that the Concurrent model achieves a peak TPS of approximately 200,000, while the Sequential model reaches nearly 230,000 TPS. Notably, both models maintain a relatively low 95th Percentile Latency of

around 100ms until the RPS approaches 200,000. Beyond this point, as the RPS nears or exceeds the system's capacity, there is a sharp increase in 95th Percentile Latency. While the Sequential model demonstrates a higher threshold for peak Transactions Per Second (TPS), the Concurrent model exhibits a more gradual increase in latency under high load conditions, suggesting enhanced stability and consistent performance during data congestion.

An unexpected observation from our experiments is the performance of the Concurrent model without Kafka transactions, represented by black lines in the figures. Contrary to initial assumptions, this model did not outperform its counterpart with Kafka transactions. This outcome indicates that integrating Kafka transactions into our system does not introduce significant overhead in scenarios with slight network delay. Instead, it provides a highly effective two-phase commit (2PC) mechanism that maintains system efficiency. This finding underscores the suitability of Kafka transactions for our distributed payment system, offering a reliable method for ensuring data consistency with minimal performance trade-offs.

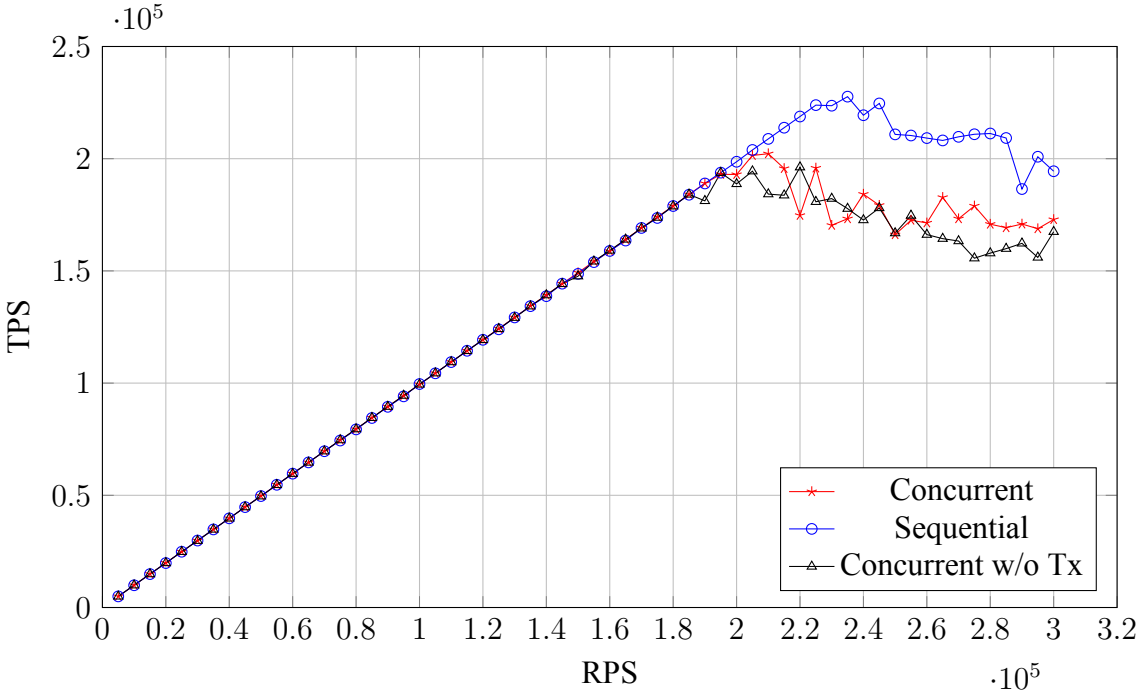


Figure 6.5: Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS)

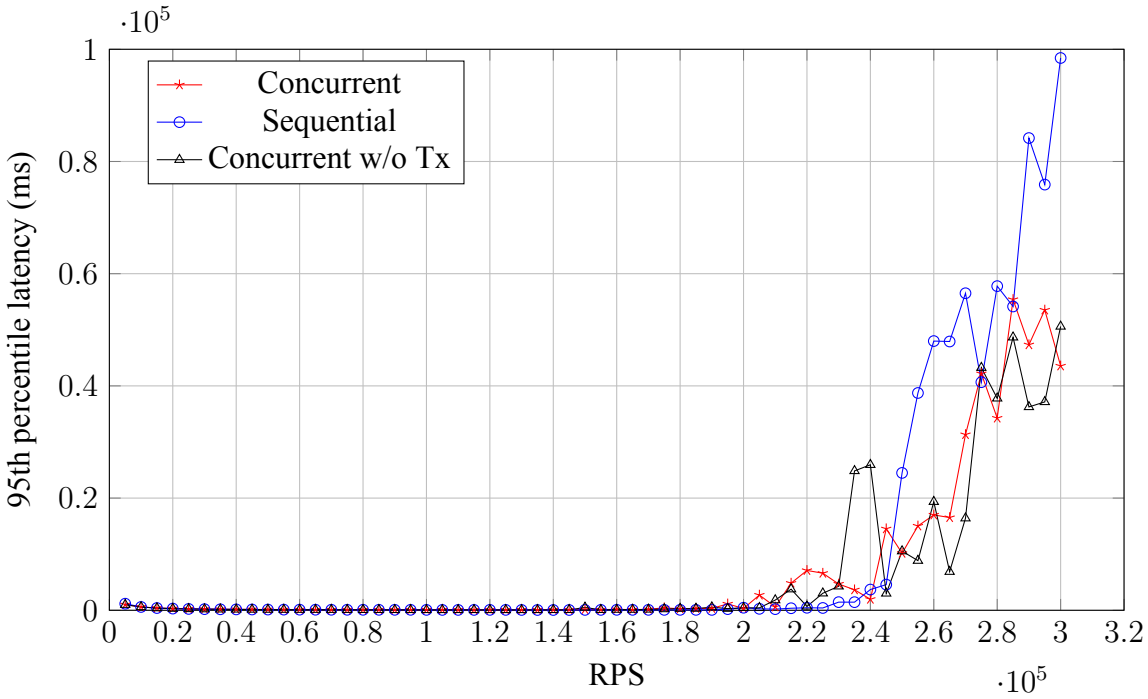


Figure 6.6: Changes in 95th Percentile Latency with Increasing Requests Per Second (RPS)

In the previous RPS tests, we ran a Kafka server, two processors, and a transaction generator on both machines. However, we questioned whether the resources used by the Kafka server and transaction generator would slow down the processors.

To find the answer, we designed another test that isolated the transaction generator on a third machine. Specifically, the original two machines continued running a Kafka server and two processors, while the third machine ran the transaction generator.

If separating one of the three components onto another machine does not change the results, it suggests that while the issue might be potential, it will not occur within the scale of our experiments. Since the Kafka server and processors can exchange messages through localhost, it is better not to separate them. Thus, we chose to isolate the transaction generator.

The results, depicted in Figure 6.7 and 6.8, show that the TPS declined after the RPS exceeded 200,000, consistent with the original RPS tests. The tests indicate that isolating the transaction generator does not increase throughput in our case, suggesting that the processors are not limited in our setup.

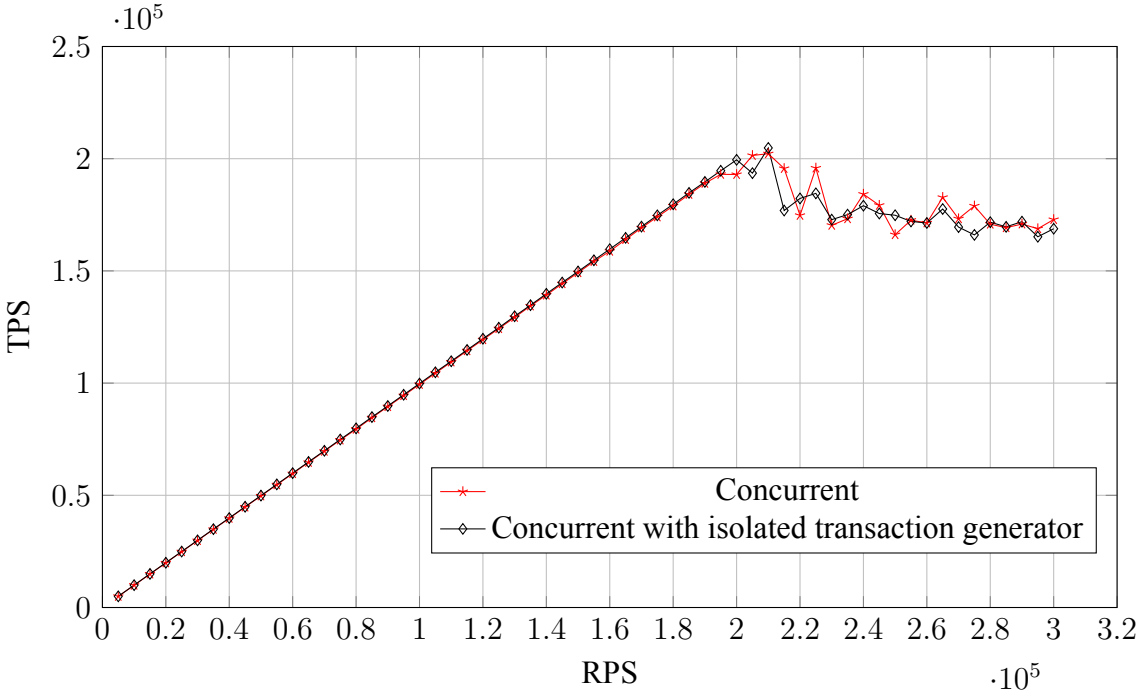


Figure 6.7: Impact of Isolated Transaction Generator for Concurrent Model

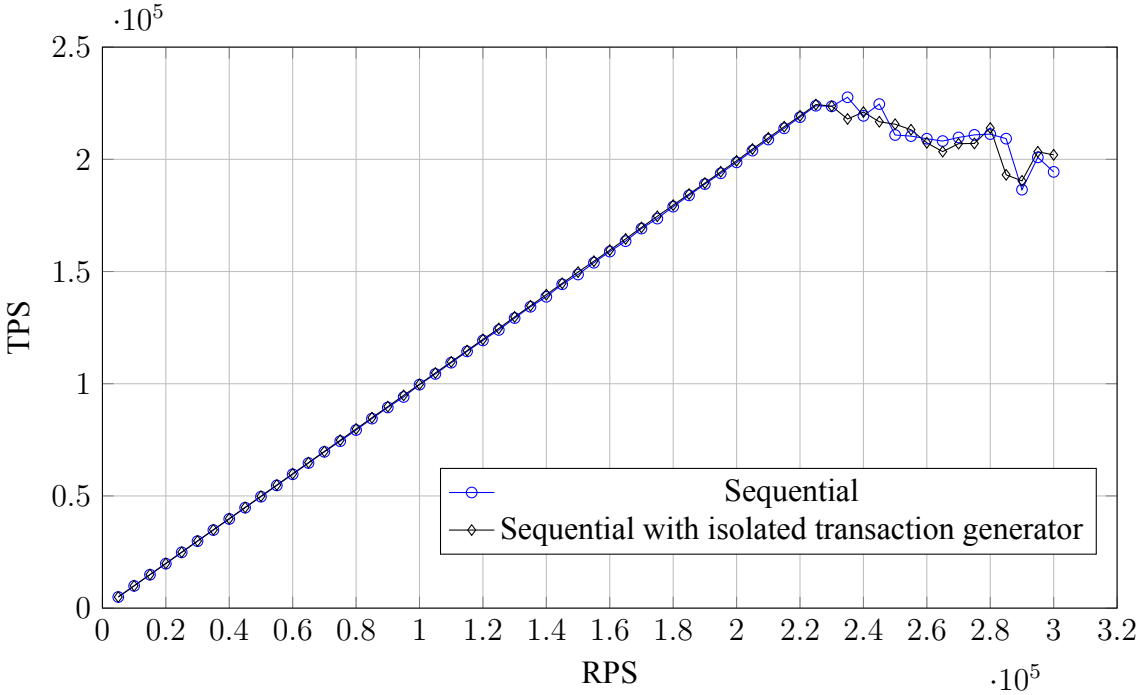


Figure 6.8: Impact of Isolated Transaction Generator for Sequential Model

6.4 The Impact of Additional Network Latency

Given the minimal network delays typical of local area networks, which formed the basis of our testing environment, our system demonstrated commendable performance. However, it's essential to acknowledge the evolving landscape of distributed systems, which increasingly favor geographically dispersed servers. This evolution naturally introduces more significant network latency, fundamentally altering system dynamics.

To adapt our experiments to more closely mirror real-world operational conditions, we simulated network delays by introducing latency directly on the network interface card (NIC). This method, while effective for our purposes, does come with limitations. Specifically, because connections to the Kafka broker—even those originating from the same machine—rely on the NIC, the introduced delay is compounded, affecting the transmission multiple times.

Despite this methodological constraint, the primary objective of these simulations was not to perfectly replicate every aspect of real-world network conditions but rather to discern the general trends and behaviors of our system under increased latency. The additional delays, although applied in a somewhat exaggerated manner, provided valuable insights into how our system's performance scales with increasing network latency. These insights are crucial for understanding potential performance bottlenecks and designing systems that are resilient to the challenges posed by geographical distribution and the inherent network delays it brings.

6.4.1 the Impact of Block Size with Additional Network Latency

Optimizing the parameters for data transmission between clients and Kafka brokers can significantly reduce the frequency of data exchanges, enhancing system efficiency.

Our prior experiments have demonstrated that both the Kafka parameters and the block size exhibit limitations only when set too low.

Delving deeper into the impact of network conditions on system performance, we introduced a simulated network delay of 100ms across all transmissions. By adjusting the maximum poll bytes to a value of two to the twentieth power, the system's stability was confirmed when the parameter was set sufficiently large, aligning with our expectations.

Figure 6.9 and 6.10 illustrate the effects of increasing block sizes under a simulated network delay. Conducting the experiment with an RPS of 100 and incrementally raising the block size from 10 to 500, we observed expected performance outcomes up to a block size of 100. Similar to previous tests, both with and without simulated delay, system capacity is constrained by excessively small block sizes.

For the concurrent model, performance stabilizes with block sizes exceeding 70, and for the sequential model, stability is achieved beyond a block size of 100. With the 95th percentile latency increasing, transaction processing remains consistent, successfully handling nearly all incoming payments. The concurrent model exhibited a minimum 95th percentile latency of approximately 4 seconds, while the sequential model showed around 5 seconds.

In a scenario with two aggregators operating on separate machines and an RPS of 100 across two account groups, each aggregator processes 25 records per second for each group. Theoretically, once the block size exceeds 25, each aggregator should transmit blocks for each group every second. However, performance is initially limited by small block sizes, particularly under the 100ms simulated delay, reaching optimization around a block size of 100. Beyond this point, although the Transactions Per Second (TPS) remains stable, the time required for aggregation progressively increases, eventually reaching the 10-second timeout limit.

This experiment yields two key insights: First, regarding network delay, we ascertain

that with a network delay of 100ms and an RPS not exceeding the system's maximum throughput, the 95th percentile latency can be maintained at approximately 5 seconds, and even lower in the concurrent model.

The second insight, a serendipitous discovery, is that based on the initial finding, the aggregator's timeout should ideally be configured to below 5 seconds if the anticipated network delay does not exceed 100ms. This configuration could be further optimized based on assurances that the network delay will remain below this threshold. By adopting such a setup, it becomes feasible to significantly increase the block size, incorporating thousands of records within a single block. This strategy enhances the system's stability under realistic network conditions, incurring minimal disadvantages. Such a configuration not only optimizes the processing efficiency in the face of network delays but also ensures that the system remains robust and responsive, effectively managing large volumes of transactions with reduced latency.

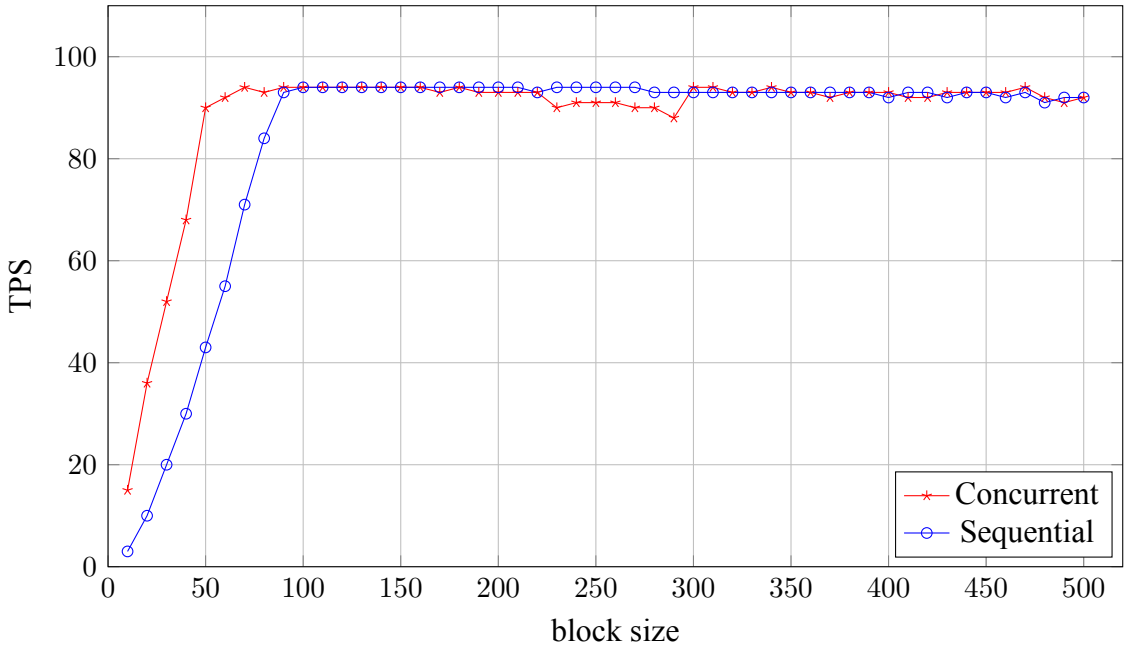


Figure 6.9: Variation of Transactions Per Second (TPS) with Increasing Block Size under Simulated Network Delay

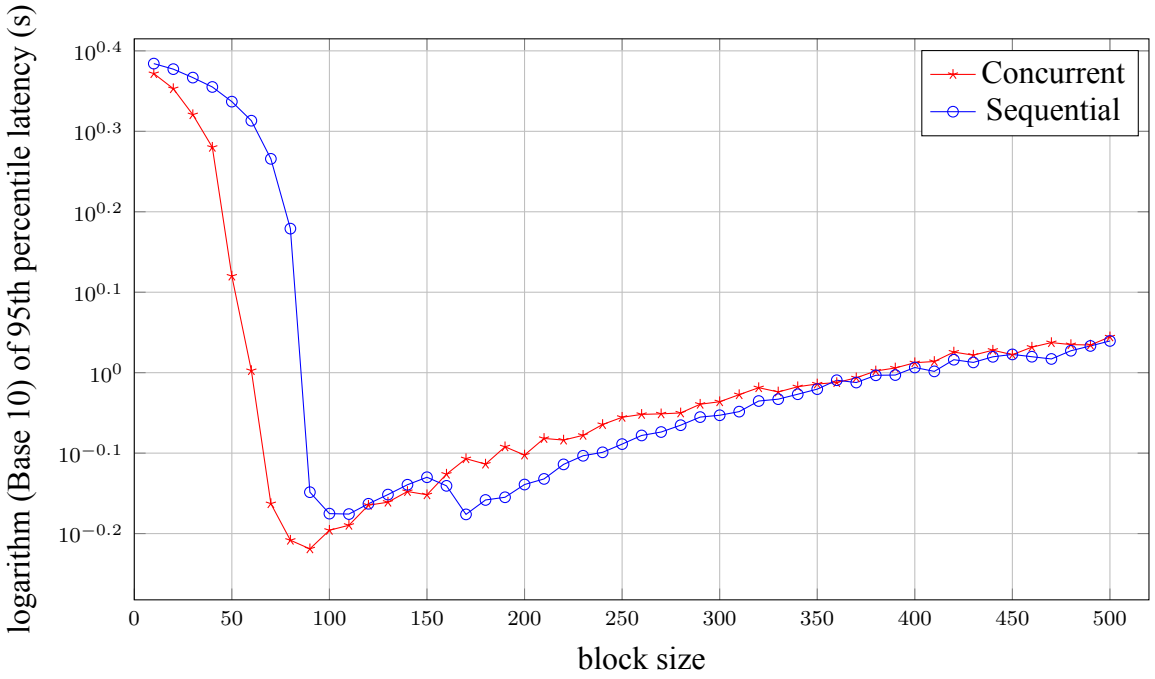


Figure 6.10: Changes in 95th Percentile Latency on a Logarithmic Scale with Increasing Block Size under Simulated Network Delay

6.4.2 The Impact of RPS with Additional Network Latency

We further evaluated the models, including the variant of the Concurrent model that operates without Kafka transactions, under a simulated network delay of 100ms. The RPS was varied from 500 to 20,000 to observe the impact on system performance. The findings from these tests are illustrated in Figure 6.11 and 6.12.

The results indicate a noticeable performance degradation due to the implementation of Kafka transactions under these conditions. This outcome is expected given that Kafka transactions embody a two-phase commit (2PC) protocol, necessitating multiple data transmissions between Kafka brokers. Despite the impact on performance, the use of Kafka transactions is integral for ensuring system consistency. These experiments aim to quantify the performance implications of Kafka transactions, especially in the context of network delays.

The primary focus of these experiments is the comparison between the Concurrent and Sequential models. Notably, once the RPS exceeds 7,000, the Transactions Per Second (TPS) for the Concurrent model consistently remains more than double that of the Sequential model, coupled with lower latency. This performance disparity underscores the Concurrent model's superior efficiency and its ability to handle higher volumes of transactions with greater speed and less delay under massive network delay.

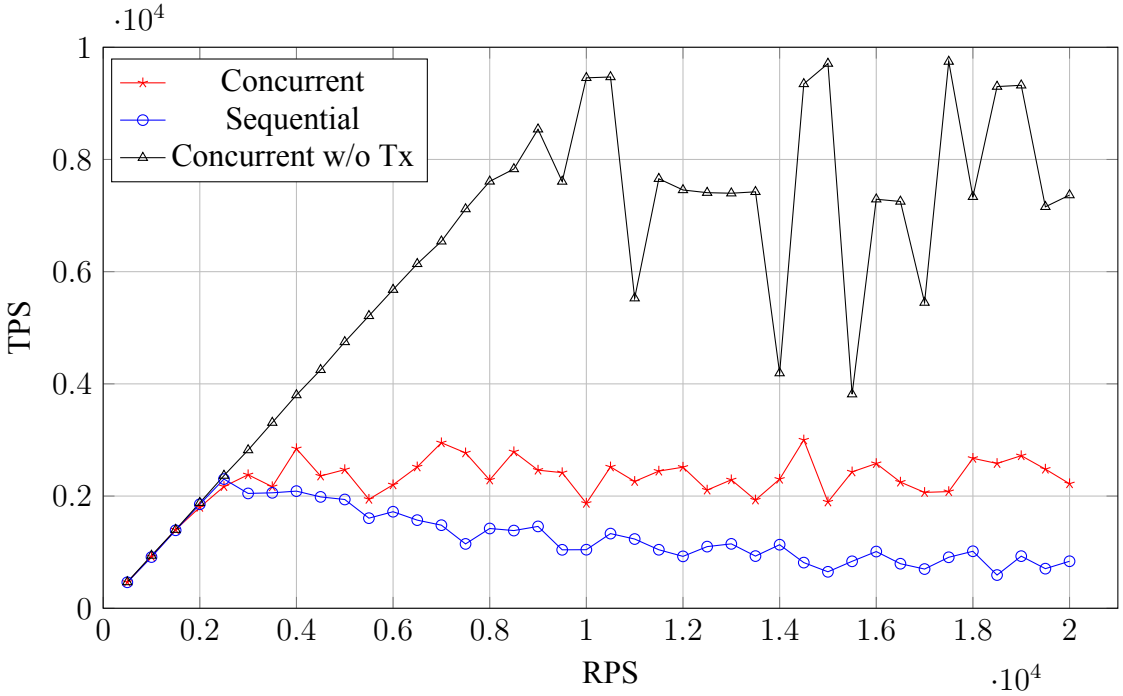


Figure 6.11: Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) under 100ms network delay

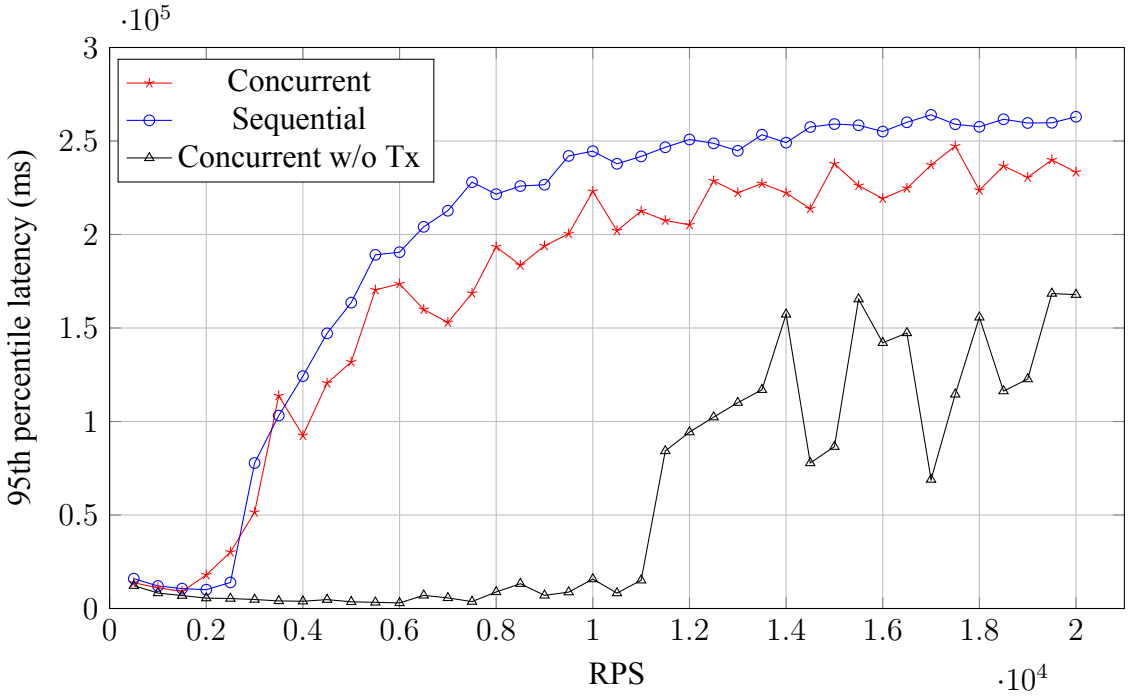


Figure 6.12: Changes in 95th Percentile Latency with Increasing Requests Per Second (RPS)under 100ms network delay

6.4.3 The Impact of Increasing Network Latency

Finally, we evaluate the impact of network latency in a reasonable block size of 500 and a constant RPS set at 100,000 with increasing network delay. We observed the impact of incrementally increasing network delays on both Transactions Per Second (TPS) and the 95th Percentile Latency. The outcomes of these experiments are depicted in Figure 6.13 and 6.14. As anticipated, an increase in network delay led to a decrease in TPS and a rise in latency, aligning with expectations that network performance significantly influences system throughput and response times.

Notably, the Concurrent model demonstrated superior performance under these conditions. Its resilience in the face of escalating network delays suggests an inherent robustness and efficiency, particularly in managing and mitigating the adverse effects of increased latency on system operations. This enhanced performance of the Concurrent

model underlines its suitability for environments where network conditions may vary or degrade, showcasing its ability to maintain higher levels of throughput and lower latency in comparison the Sequential model.

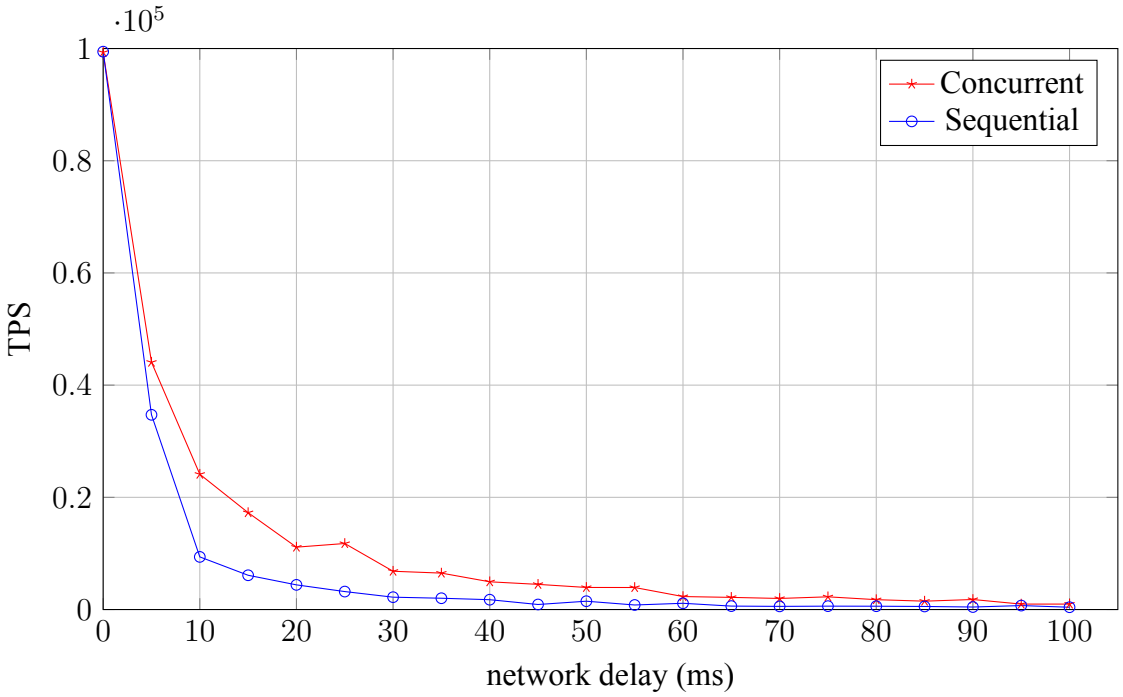


Figure 6.13: Variation of Transactions Per Second (TPS) with Increasing Network Delay

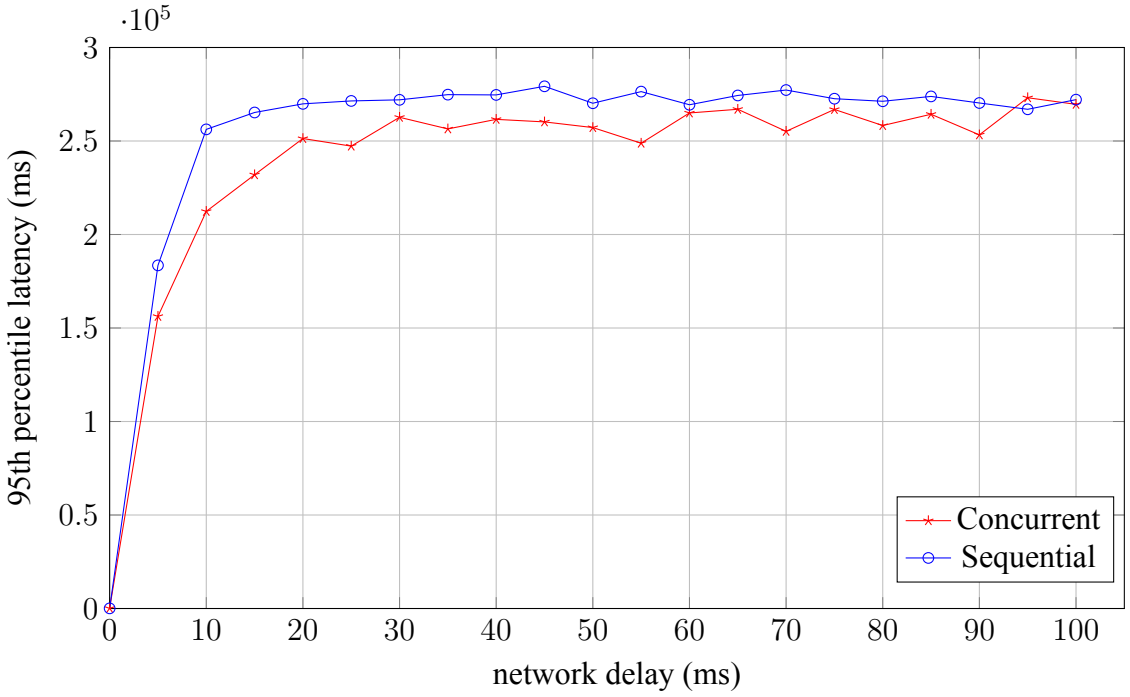


Figure 6.14: Changes in 95th Percentile Latency with Increasing Network Delay

6.5 Scalability

Earlier phases of our experiments provided insights into the throughput limitations and resilience of our systems, utilizing two identical high-performance machines. Recognizing that real-world distributed systems often comprise a heterogeneous mix of hardware, we progressed to scalability tests that integrate both high-end machines used in previous experiments and two lower-specification machines. This approach allows us to evaluate system performance in a more complex and varied hardware environment, reflecting a scenario closer to actual deployment conditions.

Through scalability tests that incorporate both high-end machines used in previous experiments and two lower-spec machines, we aim to understand how a mixed hardware environment affects our system's performance. We gradually increase the Requests Per Second (RPS) from 5,000 to 300,000 to observe the impact on Transactions Per Second

(TPS). This approach helps us gauge the scalability and adaptability of our systems as they grow in size and complexity.

6.5.1 Scenario 1: Two High-End Machines

In the set of tests, the performance outcomes of the two models were closely aligned, much like the results from previous experiments. However, the Sequential model displayed a slight advantage in performance over the Concurrent model. The results are presented in Figure 6.15.

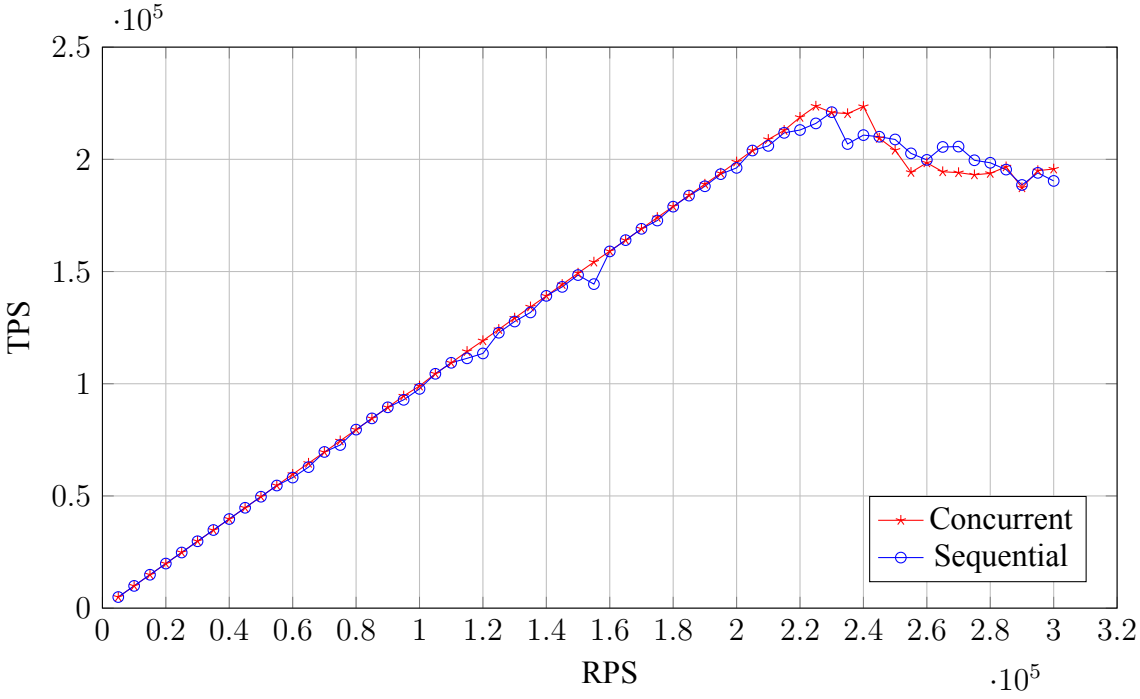


Figure 6.15: Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using Two High-End Machines

6.5.2 Scenario 2: Two Lower-Spec Machines

When utilizing the two lower-spec machines, the highest Transactions Per Second (TPS) achieved did not exceed 40,000, in contrast to the tests conducted with two high-end machines, where the TPS surpassed 200,000. The systems became overloaded when

the Requests Per Second (RPS) exceeded 40,000, resulting in a decline in throughput. Notably, under these overloaded conditions, the Concurrent model outperformed the Sequential model, a trend that was also observed in the previous tests with high-end machines. However, the earlier onset of system overload in this set of tests with lower-spec machines made the Concurrent model's superior performance under stress more pronounced, highlighting its relative efficiency and resilience in handling system overloads. The results are presented in Figure 6.16.

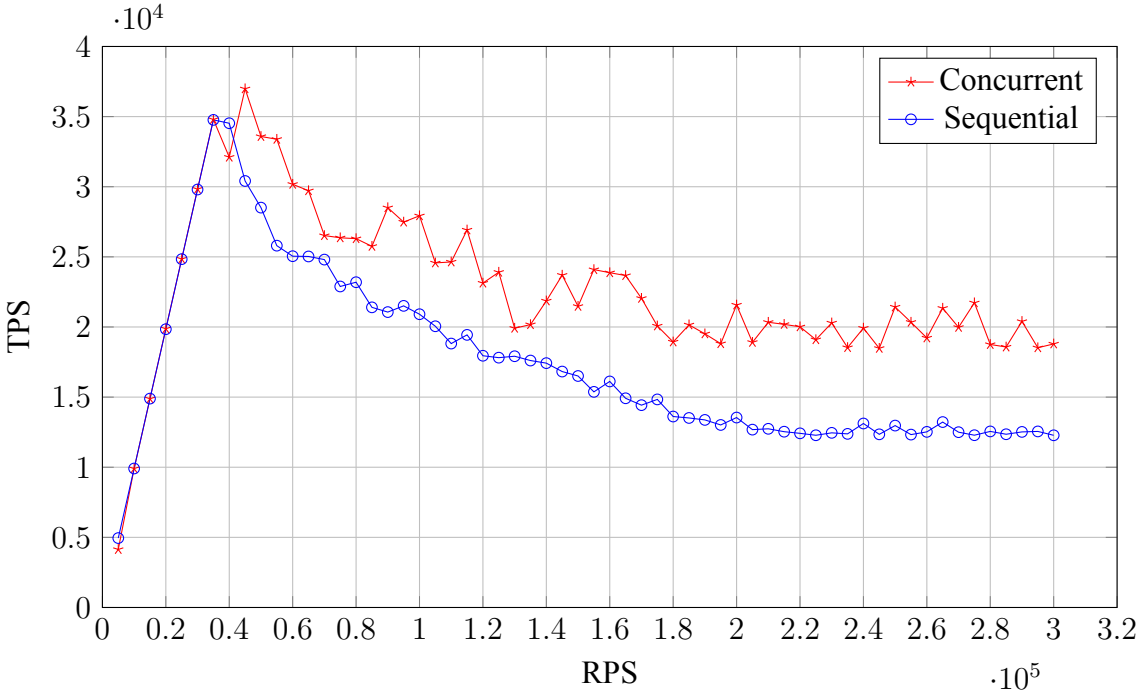


Figure 6.16: Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using Two Lower-Spec Machines

6.5.3 Scenario 3: Pairing a High-End with a Lower-Spec Machines

Upon establishing the TPS (Transactions Per Second) limits for configurations utilizing two high-end machines (around 200,000 TPS) and two lower-spec machines (under 40,000 TPS), we anticipated that pairing a high-end machine with a lower-spec machine would yield a TPS limit falling between these two benchmarks. Contrary to expectations,

the results, as illustrated in Figure 6.17, indicated a modest improvement in TPS limit to approximately 45,000 when using the mixed configuration, an outcome that, while surpassing the performance of two lower-spec machines, fell short of our projections.

In this mixed-machine setup, the Concurrent model demonstrated a generally higher TPS after the system reached its overload threshold, nearly doubling the performance compared to the configuration with two lower-spec machines. However, this configuration also exhibited more pronounced fluctuations in performance. The limited improvement in TPS and the observed variability in system performance can be attributed to the arrangement of data processing tasks across the disparate hardware. This aspect, alongside the detailed analysis of hardware configurations involving all four machines, will be discussed further, highlighting the impact of mixed hardware environments on distributed system performance and efficiency.

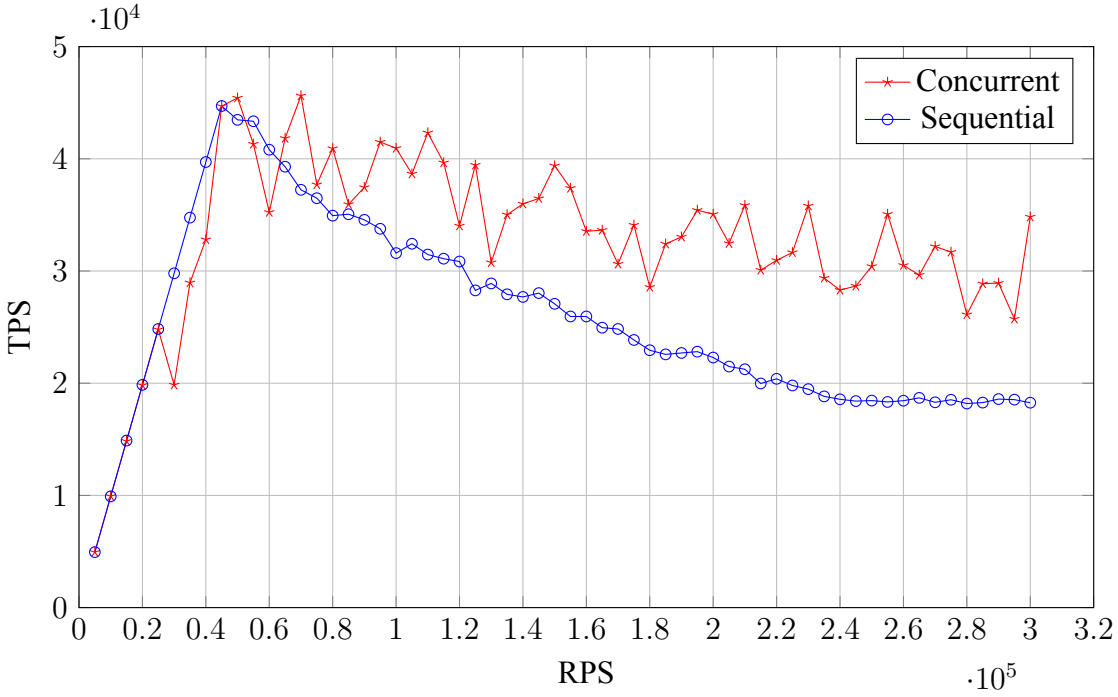


Figure 6.17: Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using a High-End and a Lower-Spec Machine

6.5.4 Scenario 4: Integrating Two High-End and Two Lower-Spec Machines

In a configuration involving two high-end machines and two lower-spec machines, the TPS limit approached 80,000, as shown in Figure 6.18. For the lower-spec machines, the addition of high-end machines resulted in a higher TPS limit, demonstrating the benefit of integrating more powerful hardware. Conversely, from the high-end machines' viewpoint, the system's performance was significantly limited by the inclusion of the lower-spec machines. Additionally, this configuration led to more pronounced fluctuations in performance for both the Concurrent and Sequential models.

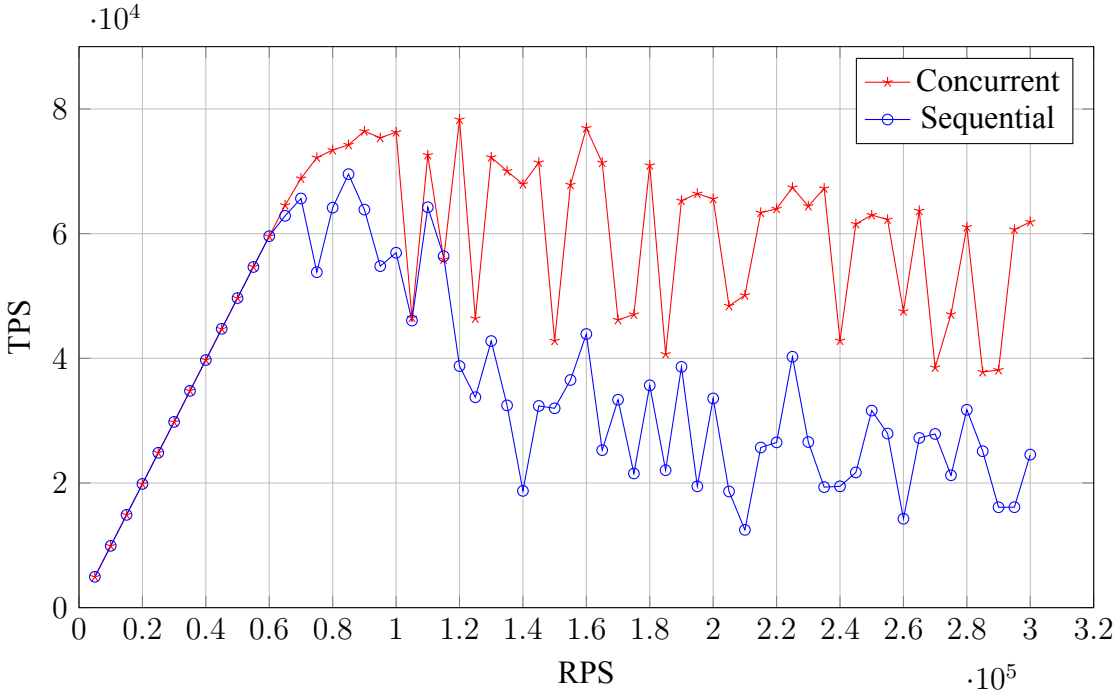


Figure 6.18: Variation of Transactions Per Second (TPS) with Increasing Requests Per Second (RPS) Using Two High-End and Two Lower-Spec Machines

6.5.5 Analysis of Four Scenarios

Figure 6.19 showcases the performance of the Concurrent model across four different machine configurations within a single graph. It reveals that the configuration combining two high-end with two lower-spec machines offers more stable results compared to the mixed configurations, which exhibit greater fluctuations. Moreover, it is noted that the performance of the two high-end plus two lower-spec configuration does not surpass that of the two high-end machines alone.

This observation does not imply that incorporating lower-spec machines into the system is inherently detrimental or incapable of enhancing system performance. However, the experimental setup, which involved randomly generating data into two or four different groups of accounts served by validators on each machine, contributed to some unexpected results for two main reasons:

Underutilization of High-End Machines: In our experiments, high-end machines processed fewer payments. For instance, in tests with a 200,000 RPS, each high-end machine in the two high-end configuration was expected to handle approximately 100,000 original payments per second and a similar number of credit payments subsequently. In contrast, in the mixed configuration of two high-end and two lower-spec machines, the high-end machines processed only about 50,000 original payments per second and were anticipated to handle the same number of credit payments later. This setup did not fully leverage the processing capabilities of the high-end machines.

Reduced Credit Payment Consumption on High-End Machines: Following the same 200,000 RPS example, the expectation that each high-end machine would process 50,000 credit payments per second in the mixed configuration was not met. Since every payment involved transactions between groups of accounts, and each validator was dedicated to a single group, the distribution of credit payments was evenly spread among the remaining

validators. Therefore, in the mixed configuration, the flow of credit payments from one high-end machine to another was significantly reduced to one third of 50,000. Even after incorporating additional inputs from the lower-spec machines, the volume will remain well below 50,000.

The disparities in processing capabilities between machine types led to traffic congestion and uncontrollable variables, which in turn resulted in fluctuations in system performance.

In more structured scenarios, such as a real-life banking system with varying sizes of banks and transaction volumes, strategically allocating suitable machines to manage specific groups of accounts could theoretically prevent system slowdowns by avoiding machine overloads. However, designing experiments to simulate such complex real-world conditions goes beyond the scope of our current evaluation, marking the conclusion of our experimental analysis.

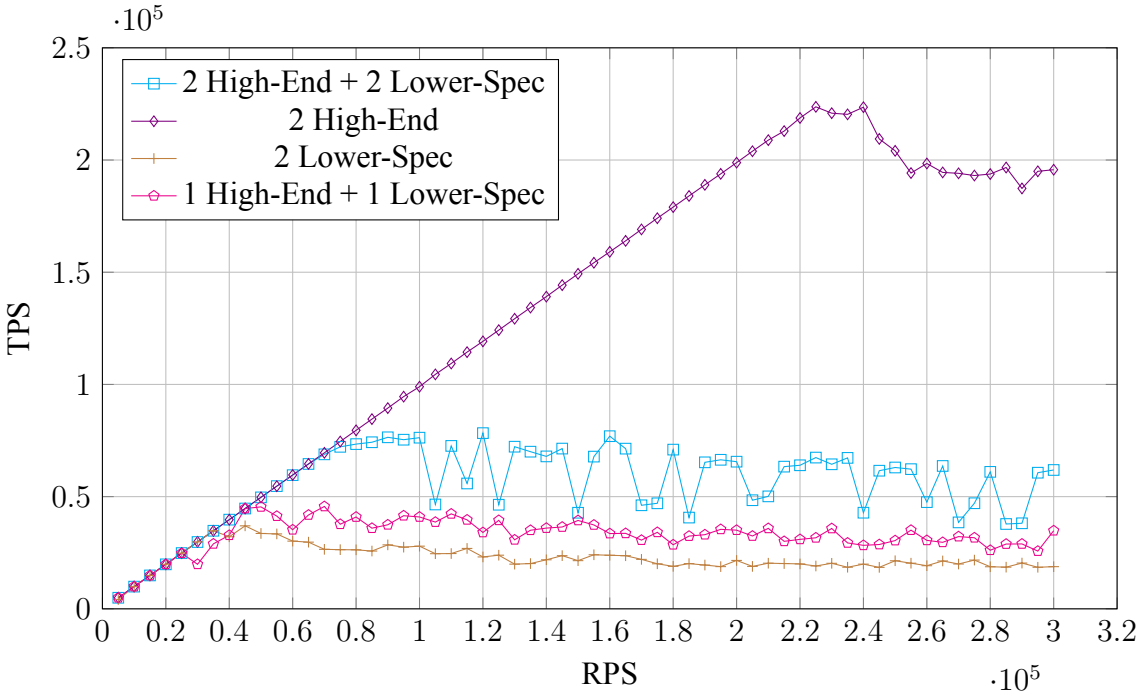


Figure 6.19: Comparative Analysis of Transactions Per Second (TPS) Variation with Increasing Requests Per Second (RPS) Across Different Machine Configurations

7 Related Work

The focal point of our research is the Central Bank Digital Currency (CBDC), which we will discuss at the outset of this chapter. The representation of user funds stands out as a pivotal and distinct concept within our systems; hence, it will be the subject of the second section. The final section will address the two-phase commit protocol. Although not extensively covered in this thesis due to our reliance on Apache Kafka's inherent capabilities, its features significantly contribute to our system's design. Furthermore, the contemporary variations in approaches to the two-phase commit protocol are also deemed critical.

7.1 Central Bank Digital Currency

There are considerable ongoing researches in the realm of Central Bank Digital Currency (CBDC), from initial explorations and research to pilot programs, and in some instances, the deployment of fully operational products.

The Bank for International Settlements (BIS) has identified three primary frameworks for CBDC deployment based on the roles of intermediaries: the direct model, where the central bank issues CBDC directly to users; the two-tier model, where the central bank distributes CBDC through intermediaries who then interact with the end-users; and a hybrid model that incorporates elements of both. [14]

In the context of swift payments and real-time processing, the direct model—eliminat-

ing intermediaries—appears more fitting. Numerous central banks are already facilitating real-time and fast payment systems. [15] These platforms are engineered to expedite transactions among qualified financial entities with minimal delay. Nevertheless, such systems are not designed to manage the vast volumes typical of national retail payment systems, nor do they offer direct public access. [16, 17]

Conversely, the two-tier model is exemplified by several implementations, including China's centralized e-CNY system, which is undergoing public trials. [18, 19] Similar CBDC frameworks include the Central Bank of the Bahamas' Sand Dollar and other systems based on Bitt ' s platform, such as the Central Bank of Nigeria's eNaira and the Eastern Caribbean Central Bank's DCash. [20]

When the primary focus is not solely on fast payments and real-time systems, research tends to favor the two-tier or hybrid models. These approaches leverage the combined strengths of central banks and financial institutions to enhance efficiency, accessibility, innovation, and security. However, these models do not inherently offer scalable parallel processing capabilities.

Some projects have demonstrated the potential for linear scalability through a parallel architecture. For instance, Eesti Pank and other Eurosystem banks have experimented with a CBDC structure that tracks bill groups across parallel blockchains. [21] While this method allows for scalable growth, it necessitates external coordination for transactions involving multiple bills, as it lacks an internal mechanism to ensure transaction atomicity.

In this thesis, we propose a system based on the two-tier operational model, wherein intermediaries play a crucial role in distributing CBDC to the public. Commercial banks or designated financial institutions gather requests from the public and forward them to the central bank's main system, as detailed previously. Following transaction processing—whether approved or rejected—the intermediaries are notified to respond to their clients. This two-tier setup, which inherently delays transactions, exhibits an interesting dynamic

in our model. Although only eventual consistency is maintained—far from a real-time system—the intermediaries are able to immediately aware of transaction completion as soon as the initial phase (the remittance part) is executed. This approach allows for horizontal scalability, courtesy of the two-tier model.

As previously highlighted, the two-tier models include centralized systems such as China's e-CNY. Our approach, while not overly focused on stringent security or regulatory constraints, prioritizes maximizing efficiency and ensuring consistency through a distributed system design that differentiates from other two-tier models. Notably, our design, rooted in the two-tier operational framework, also explores opportunities to reduce latencies. By providing intermediaries with dedicated servers (e.g., Kafka brokers) and specialized CBDC processors (including aggregators and validators), we aim to mitigate some inherent limitations of the two-tier model, enhancing the system's overall performance and responsiveness.

7.2 Fund Representation

As outlined in Chapter 2, commercial systems typically represent user funds through either an account-based or UTXO (Unspent Transaction Outputs)-based model. The account-based model, a traditional approach, directly represents funds within accounts, a method not only prevalent in established centralized payment systems but also adopted by some cryptocurrencies in recent years. Cryptocurrencies like Ethereum [22] have innovated on this model by utilizing public key cryptography and digital signatures for authorization, rather than traditional passwords, demonstrating the model's adaptability to distributed systems.

Conversely, the UTXO model, initially popularized by Bitcoin [23], represents user funds as unspent transaction outputs. This method has significantly influenced not just

the cryptocurrency domain but also the development of CBDC systems. Projects such as China's e-CNY [18, 19]and the Federal Reserve Bank of Boston's Project Hamilton [1] illustrate the adoption of the UTXO model for representing user funds within the realm of CBDCs.

In this thesis, we express a preference for the account-based model due to its inherent simplicity. However, adhering to traditional transaction processing methods significantly limits the level of achievable parallelism. To navigate this challenge, our system generates credit post-transaction validation, emulating aspects of UTXO-based models. This method positions the representation of user funds closely with accounts in a strict classification sense. Nonetheless, the intricacies of our system's operations draw parallels with the UTXO model, thus merging the benefits of both types of fund representation.

A related master's thesis, written by Chiang Pin-Hsuan [24], for which we shared the prototype of the system and scattered its development to build different systems, used a similar approach to separate transactions in the banking system. The biggest difference is that Chiang's version sends the credit records to an individual topic, and concerned validators poll from the topic every few seconds. However, this approach delays consistency and requires rebuilding connections to the Kafka server, which allows us to process tens or hundreds of thousands of payments, in every cycle.

Our credit model's strategy also mirrors the application of sharding [25, 11] in Monoxide, a blockchain system renowned for its scalability through the full sharding of its consensus protocol while maintaining a security level on par with Bitcoin and Ethereum, as examined by Jiaping Wang and Hao Wang [26]. Unlike our approach, which focuses on enhancing transaction processing efficiency, Monoxide leverages the account-based model to shard transactions for a different purpose—scaling out blockchains with asynchronous consensus zones. This demonstrates a shared interest in combining model attributes to overcome inherent limitations, though our motivations and applications diverge. The sys-

tem is similar to our sequential model, which sends the received part of the payments back into the queue. However, the entire system lacks 2PC (two-phase commit) protection, meaning it sometimes needs to roll back some payments to avoid incorrect results.

7.3 Two-phase Commit

In the domain of transaction processing, employing a distributed algorithm to ensure atomic commitment is fundamental, with the two-phase commit (2PC) protocol often playing a central role. However, the architecture of 2PC implementations has evolved to cater to specific requirements across various systems. [27]

For instance, Natto [28] and Carousel [29], both geo-distributed database systems designed for low-latency operations, employ variants of the two-phase commit protocol. Similarly, Google's Spanner utilizes a hybrid approach that combines two-phase commit with a replicated state machine [30], specifically Raft [31], although it does not support general SQL operations. Project Hamilton, an initiative by the Federal Reserve Bank of Boston [1], introduces an unconventional form of two-phase commit that notably omits the need for rollback capabilities.

In this thesis, our system utilizes an unconventional two-phase commit mechanism within the Apache Kafka [6] framework, reducing the reliance on acknowledgments (acks) by registering the Kafka producer to a concealed topic within the Kafka ecosystem, mirroring the methodology employed by Spanner. Without the accelerated versions of the two-phase commit protocol, our tests, particularly under simulated network delays, would yield substantially poorer outcomes.

8 Conclusion

We have developed two models for a distributed payment system leveraging Apache Kafka, diverging from approaches tailored for central bank digital currencies[1]. Our design intentionally restricts direct client access to the system, favoring inputs from specific, authorized sources such as banks' servers rather than from individual bank account holders. This approach allows us to minimize external threats, thereby concentrating our efforts on enhancing system performance and scalability. By employing Kafka's architecture and adopting a credit model to represent users' funds, we have constructed two distinct models.

Through rigorous testing focused on performance, stability, and scalability, the Concurrent model has emerged as superior. Remarkably, this model is capable of processing up to 200,000 transactions per second (TPS) with just two laptops managing the system, all while maintaining consistency. It demonstrates resilience against network latency variations and robustness in addressing severe issues, such as node disconnections or failures, leveraging Kafka's inherent capabilities.

Our exploration represents a pioneering effort in reconciling account balances with transactional data outside the blockchain domain[26], aiming for eventual consistency within the banking system. This thesis marks an initial foray into integrating fragmented transactional data to achieve coherent account balance representation in banking systems not reliant on blockchain technology.

Reference

[1] J. Lovejoy, C. Fields, M. Virza, T. Frederick, D. Urness, K. Karwaski, A. Brownworth, and N. Narula, "A high performance payment processing system designed for central bank digital currencies," *Cryptology ePrint Archive*, 2022.

[2] S. Allen, S. Čapkun, I. Eyal, G. Fanti, B. A. Ford, J. Grimmelmann, A. Juels, K. Kostiainen, S. Meiklejohn, A. Miller, *et al.*, "Design choices for central bank digital currency: Policy and technical considerations," tech. rep., National Bureau of Economic Research, 2020.

[3] R. Auer and R. Böhme, "The technology of retail central bank digital currency," *BIS Quarterly Review, March*, 2020.

[4] R. Auer, G. Cornelli, and J. Frost, "Rise of the central bank digital currencies: drivers, approaches and technologies," 2020.

[5] J. Zahnentferner, "Chimeric ledgers: Translating and unifying utxo-based and account-based cryptocurrencies," *Cryptology ePrint Archive*, 2018.

[6] N. Garg, *Apache kafka*. Packt Publishing Birmingham, UK, 2013.

[7] B. Bejeck, *Kafka Streams in Action: Real-time apps and microservices with the Kafka Streams API*. Simon and Schuster, 2018.

[8] M. Seymour, *Mastering Kafka Streams and ksqlDB*. O'Reilly Media, 2021.

[9] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, "Zookeeper: wait-free coordination for internet-scale systems.," in *USENIX annual technical conference*, vol. 8, 2010.

[10] G. Danezis and S. Meiklejohn, "Centrally banked cryptocurrencies," *arXiv preprint arXiv:1505.06895*, 2015.

[11] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pp. 17–30, 2016.

[12] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[13] G. Pîrlea and I. Sergey, "Mechanising blockchain consensus," in *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pp. 78–90, 2018.

[14] "CBDCs: an opportunity for the monetary system," *BIS Annual Report Economic Report*, p. 65–91, 2021.

[15] B. für Internationalen Zahlungsausgleich Committee on Payments and M. Infrastructures, *Fast payments: Enhancing the speed and availability of retail payments*. Bank for International Settlements, 2016.

[16] *TARGET Instant Payments Settlement user requirements*. European Central Bank, 2017.

[17] *T2-T2S consolidation user requirements document for T2-RTGS component*. European Central Bank, 2018.

[18] J. Jiang and K. Lucero, "Background and implications of china's e-cny," *U. Fla. JL & Pub. Pol'y*, vol. 33, p. 237, 2022.

[19] Y. Qian, "Technical aspects of cbdc in a two-tiered system," in *presentation materials presented at ITU Workshop on Standardizing Digital Fiat Currency (DFC) and its Applications, New York City, United States*, pp. 18–19, 2018.

[20] D. W. Patrick and T. Lyle, "Central bank digital currency: Caribbean pathways," *how the challenges of today prepare the ground for tomorrow*, p. 340, 2022.

[21] *Work stream 3: A new solution –blockchain & eID*. Eesti Pank, 2021.

[22] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[23] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Decentralized business review*, 2008.

[24] P.-H. Chiang, "An empirical study on the scalability of kafka in a scenario of online payments," Master's thesis, National Chengchi University, 2023.

[25] A. Adya, D. Myers, J. Howell, J. Elson, C. Meek, V. Khemani, S. Fulger, P. Gu, L. Bhuvanagiri, J. Hunter, *et al.*, "Slicer:{Auto-Sharding} for datacenter applications," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pp. 739–753, 2016.

[26] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *16th USENIX symposium on networked systems design and implementation (NSDI 19)*, pp. 95–112, 2019.

[27] J. N. Gray, "Notes on data base operating systems," *Operating systems: An advanced course*, pp. 393–481, 2005.

[28] L. Yang, X. Yan, and B. Wong, "Natto: Providing distributed transaction prioritization for high-contention workloads," in *Proceedings of the 2022 International Conference on Management of Data*, pp. 715–729, 2022.

[29] X. Yan, L. Yang, H. Zhang, X. C. Lin, B. Wong, K. Salem, and T. Brecht, "Carousel: Low-latency transaction processing for globally-distributed data," in *Proceedings of the 2018 International Conference on Management of Data*, pp. 231–243, 2018.

[30] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, *et al.*, "Spanner: Google ' s globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 1–22, 2013.

[31] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *2014 USENIX annual technical conference (USENIX ATC 14)*, pp. 305–319, 2014.